Wind power forecasting under corrupted SCADA for edge deployment: sparse MoE with feasibility guard

Junyu Li[a], Juntao Du[a,*]

[a]*School of Statistics and Applied Mathematics, Anhui University of Finance and Economics, Bengbu, 233030, China*

Abstract

In this study on Wind (CN), we examine forecasting under corrupted SCADA telemetry with batch-1 edge limits on latency and memory, and observe that clean-benchmark ranking can break down in dispatch-coupled operation on Wind (CN). We propose Ultra-LSNT, a sparse MoE forecaster, together with FG-MoE, a feasibility guard that suppresses physically invalid outputs during corrupted-input inference with minimal additional overhead in batch-1 deployment settings. On Wind CN, Ultra-LSNT retains positive skill in the severe Gaussian slice used in this study, with $\sigma_{\text{eff}} = 0.60$ and $R^2 = 0.7263$, and FG-MoE reduces over-rated violations from 0.0550% to 0.0092% against a matched no-guardrail backbone. Under a fixed affine mapped-dispatch protocol, Ultra-LSNT lowers L1 operating cost and fallback frequency versus DLinear, with cost reduction −33.96% and fallback days reduced from 75 to 29; ranking remains interface-sensitive, with mean Spearman = 0.60 and Top-1 consistency = 0.

Keywords: Wind power forecasting, Economic dispatch, SCADA imperfections, Edge computing, Robustness, Wind power integration

1. Introduction

Wind-power forecasting for grid operation is often limited by three coupled constraints: imperfect SCADA telemetry, tight edge-controller inference budgets, and physical feasibility requirements. Drift, quantization, communication dropouts, and abnormal pulses are common in field data [1–4]. When these perturbations enter dispatch pipelines, unstable predictions can increase fallback scheduling, curtailment, and balancing pressure [5–7].

*Corresponding author.
Email addresses: ljylikezmn999@gmail.com (Junyu Li), dujuntao@aufe.edu.cn (Juntao Du)

In this study, we observe that clean-benchmark ranking may not transfer reliably to corrupted SCADA operation; the severe-slice evidence is summarized in Table 2 and Figure 4. Dense deep sequence models achieve strong clean-data accuracy but typically rely on fixed computation pathways at inference [8–12]. Decomposition-optimization hybrids can also be highly accurate, yet their iterative decomposition and search loops are expensive for batch-1 controller-side use [13–17]. Time-series foundation models improve transfer capability, but their resource assumptions remain closer to cloud-scale deployment than to Remote Terminal Unit / Programmable Logic Controller (RTU/PLC) constraints on typical on-site controller hardware [18–22]. Prior work has used turbine power-curve conventions and test-oriented feasibility checks to limit non-physical outputs [23, 24]. Under corrupted inputs and batch-1 edge deployment, however, lightweight feasibility control coupled with conditional routing remains underexplored.

We build Ultra-LSNT (Ultra-Lightweight Sparse Neural forecasTing) for this setting with Top-K sparse routing, so computation can focus on difficult corrupted segments instead of a dense always-on path. We couple it with FG-MoE, which enforces turbine-envelope consistency through a lightweight forward-pass feasibility guard.

Contributions: First, we develop a sparse-routing forecaster with an integrated feasibility guard for short-term wind prediction under corrupted telemetry. Second, we evaluate robustness, physical validity, edge efficiency, and dispatch-coupled outcomes in one shared protocol [25–28]. Third, we report where conclusions are stable and where they are interface-sensitive, and summarize these boundaries in Discussion and the claim-scope appendix (Appendix F).

2. Related work and theoretical background

Wind power forecasting sits at the intersection of meteorology, statistical learning, deep sequence modeling, and power-system operations. This section positions the methodological gap addressed in our deployment setting.

2.1. The representational-operational gap in sequence modeling

Dense attention models and state-space models provide strong temporal representation on curated benchmarks [8, 9, 11, 12]. Hybrid decomposition-optimization pipelines remain competitive in many energy studies [14–16]. Under corrupted telemetry, however, both families

often rely on fixed inference pathways and leave less room to reallocate computation to difficult segments.

Under drift, quantization, and intermittent dropouts, we empirically observe a larger clean-to-severe performance gap in fixed-pathway models in our benchmark, especially in always-on batch-1 inference settings; see Table 2 and Figure 4. TSFMs such as Chronos and Moirai raise transfer upper bounds [18–20], but are used here as capability references rather than controller-side baselines because of their resource footprint.

2.2. SCADA hostility and the need for feasibility-guarded routing

Industrial SCADA streams are intrinsically imperfect: calibration drift, dropout, clipping, and communication jitter are common [1]. Predictors trained under clean assumptions can therefore produce trajectories that violate turbine envelopes, including over-rated outputs.

In dispatch workflows, weak output-feasibility control can increase infeasible schedules and corrective recourse. We therefore focus on two mechanisms: **(i)** conditional sparse routing that reallocates computation under corrupted telemetry, and **(ii)** feasibility-constrained output projection that suppresses physically invalid forecasts in the forward graph.

3. Methodology

Given multivariate SCADA inputs, the model builds multi-scale embeddings, routes them through sparse experts, and applies feasibility-aware output projection.

3.1. Problem setup

Given a multivariate look-back window $\mathbf{X} \in \mathbb{R}^{L\times C}$ with L historical steps and C SCADA covariates, the task is to predict an H-step active-power trajectory $\hat{\mathbf{Y}} \in \mathbb{R}^{H\times 1}$. The constrained learning problem is

$$\min_{\theta} \ \mathbb{E}\Big[\mathcal{L}_{\text{pred}}(\hat{\mathbf{Y}}, \mathbf{Y})\Big]. \tag{1}$$

$$\hat{y}_{t+h} \in \mathcal{F}_{\text{phys}}(v_{t+h}). \tag{2}$$

where $\mathcal{F}_{\text{phys}}(\cdot)$ denotes the feasible power envelope. It is parameterized by turbine specifications $(P_{\text{rated}},\ v_{\text{in}},\ v_{\text{rated}},\ v_{\text{out}})$. The construction of $\mathcal{F}_{\text{phys}}$ is given in Section 3.4.

3.2. Multi-scale decomposition and embedding

Wind-power trajectories contain low-frequency trends and high-frequency residual fluctuations. The input is decomposed as

Following decomposition–fusion practice in renewable forecasting [14, 29], we apply a moving-average operator:

$$\mathbf{X} = \mathbf{X}_{\text{trend}} + \mathbf{X}_{\text{season}}, \qquad \mathbf{X}_{\text{trend}} = \text{MA}(\mathbf{X}), \tag{3}$$

where $\text{MA}(\cdot)$ is the moving-average operator.

The residual component is embedded through parallel 1D convolutions:

$$\mathbf{E}_s = \text{Conv1D}_{k_s}(\mathbf{X}_{\text{season}}), \tag{4}$$

$$\mathbf{E}_m = \text{Conv1D}_{k_m}(\mathbf{X}_{\text{season}}), \tag{5}$$

followed by learnable fusion:

$$\mathbf{E} = \phi(\mathbf{W}_s\mathbf{E}_s + \mathbf{W}_m\mathbf{E}_m)\,, \tag{6}$$

where $\phi(\cdot)$ is a pointwise nonlinearity and $\mathbf{W}_s, \mathbf{W}_m$ are projection matrices. This decomposition-embedding block provides the input representation for sparse routing.

3.3. Sparse routing module

This module defines conditional computation through Top-K sparse routing. Our routing formulation follows the classical sparse MoE line of conditional computation and load-balanced gating, adapted here to time-series forecasting with a Jump Gate and feasibility-aware output path [30–32].

3.3.1. Routing formulation

Given the decomposition-and-embedding front-end above, the routed representation is

$$\mathbf{E} = \phi(\mathbf{W}_s\mathbf{E}_s + \mathbf{W}_m\mathbf{E}_m)\,, \tag{7}$$

where $\mathbf{E}_s, \mathbf{E}_m \in \mathbb{R}^{L\times d}$ and $\mathbf{W}_s, \mathbf{W}_m \in \mathbb{R}^{d\times d}$.

Figure 1 shows the model interface under the main Wind (CN) setting ($L = 96$, $H = 24$): decomposition, sparse Top-K routing, Jump Gate skipping, and feasibility-aware output control in one forward graph.

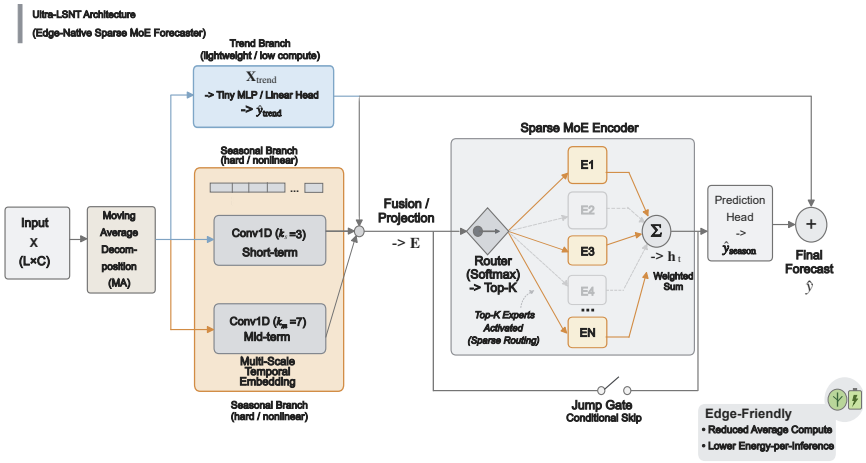

Figure 1: Ultra-LSNT architecture with decomposition, multi-scale Conv1D embedding, Top-K MoE routing, and a Jump Gate skip path.

Conditional computation (Top-K experts and Jump Gate) is coupled to an explicit feasibility-control output path rather than appended as post-processing. This coupling motivates evaluating routing and output validity together in later sections.

Frequency-aware alternatives such as VMD/EEMD can be used in the same interface [33, 34].

3.3.2. *Training objective*

Let $\mathbf{e}_t \in \mathbb{R}^d$ denote the embedding at step t. Let $\{E_i(\cdot)\}_{i=1}^N$ denote expert networks:

$$E_i(\mathbf{e}_t) = \sigma(\mathbf{W}_i \mathbf{e}_t + \mathbf{b}_i), \tag{8}$$

where $\mathbf{W}_i \in \mathbb{R}^{d\times d}$ and $\mathbf{b}_i \in \mathbb{R}^d$.

A router computes logits and probabilities:

$$\mathbf{z}_t = \mathbf{W}_r \mathbf{e}_t + \boldsymbol{\epsilon}_t, \qquad \mathbf{p}_t = \mathrm{Softmax}(\mathbf{z}_t), \tag{9}$$

where $\mathbf{W}_r \in \mathbb{R}^{N\times d}$ and $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \eta_e^2 \mathbf{I})$. The exploration scale is annealed by epoch index e:

$$\eta_e = \eta_0 \exp(-e/\tau_\eta). \tag{10}$$

Top-K routing selects $\mathcal{S}_t = \mathrm{TopK}(\mathbf{p}_t, K)$ and aggregates outputs as

$$\mathbf{h}_t = \sum_{i\in\mathcal{S}_t} p_{t,i}\, E_i(\mathbf{e}_t). \tag{11}$$

Training uses prediction loss and routing regularization.

Point forecasting is optimized by MSE:

$$\mathcal{L}_{\text{pred}} = \frac{1}{H} \sum_{h=1}^{H} \|\hat{y}_{t+h} - y_{t+h}\|_2^2 \,. \tag{12}$$

Router entropy and expert disagreement are defined as

$$H_t = -\sum_{i=1}^{N} p_{t,i} \log(p_{t,i} + \epsilon), \tag{13}$$

with $\epsilon = 10^{-8}$, and

$$D_t = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \left(\hat{y}_{t+h}^{(i)} - \bar{y}_{t+h} \right)^2 . \tag{14}$$

$$\bar{y}_{t+h} = \frac{1}{|\mathcal{S}_t|} \sum_{i \in \mathcal{S}_t} \hat{y}_{t+h}^{(i)}. \tag{15}$$

These quantities parameterize an adaptive reserve rule:

$$\sigma_{w,t} = \min\Big(\sigma_{\max}, \sigma_{\text{base}} + \alpha_H H_t + \alpha_D D_t\Big), \tag{16}$$

where σ_{base}, α_H, α_D, and $\sigma_{\max}$ are fixed coefficients. Optional probabilistic heads are

$$\mathcal{L}_{\text{pin}} = \frac{1}{|Q|H} \sum_{q \in Q} \sum_{h=1}^{H} \rho_q\Big(y_{t+h} - \hat{y}_{t+h}^{(q)}\Big) . \tag{17}$$

$$\rho_q(e) = \max(qe, (q-1)e). \tag{18}$$

$$\mathcal{L}_{\text{nll}} = \frac{1}{H} \sum_{h=1}^{H} \left(\frac{(y_{t+h} - \mu_{t+h})^2}{2\sigma_{t+h}^2} + \log \sigma_{t+h} \right). \tag{19}$$

The router load-balancing regularizer is

$$\mathcal{L}_{\text{aux}} = \sum_{i=1}^{N} \left(\bar{p}_i - \frac{1}{N} \right)^2, \qquad \bar{p}_i = \frac{1}{T} \sum_{t=1}^{T} p_{t,i}, \tag{20}$$

and the routing-module objective is

$$\mathcal{L}_{\text{base}} = \mathcal{L}_{\text{pred}} + \lambda_r \, \mathcal{L}_{\text{aux}}. \tag{21}$$

3.4. Feasibility-constrained output projection

FG-MoE adds physical constraints to the forward graph. The derivation starts from aerodynamic power conversion and then defines a SCADA-observable envelope.

A horizontal-axis turbine is bounded by

$$P_{\text{aero}}(v) = \frac{1}{2}\rho A v^3 C_p(\lambda_{\text{tsr}}, \beta), \qquad A = \pi R^2, \tag{22}$$

where ρ is air density, A is rotor swept area, v is hub-height wind speed, and C_p is the power coefficient with $0 \le C_p \le C_{p,\max} \le 16/27$. The operational envelope is parameterized by $(P_{\text{rated}}, v_{\text{in}}, v_{\text{rated}}, v_{\text{out}})$ following standard wind-turbine power-curve conventions and test practice [4, 23, 24].

When SCADA channels are incomplete (e.g., missing aerodynamic states such as β), FG-MoE uses a robust dynamic proxy for future wind speed. Define increments

$$\delta v_{t-i} = v_{t-i+1} - v_{t-i}, \qquad i = 1, \ldots, w. \tag{23}$$

Then

$$\Delta v_t^{\text{med}} = \text{median}\{\delta v_{t-i}\}_{i=1}^{w}, \tag{24}$$

$$s_t^{\text{MAD}} = 1.4826 \cdot \text{median}\{|\delta v_{t-i} - \Delta v_t^{\text{med}}|\}_{i=1}^{w}, \tag{25}$$

$$g_t = \text{clip}\Big(\Delta v_t^{\text{med}}, -\kappa s_t^{\text{MAD}}, +\kappa s_t^{\text{MAD}}\Big), \tag{26}$$

and the time-decayed proxy is

$$\hat{v}_{t+h} = v_t + \gamma_h\, h\, g_t, \qquad \gamma_h = \exp(-h/\tau_v). \tag{27}$$

Using $\hat{v}_{t+h}$, the envelope is

$$P_{\text{env}}(\hat{v}) = \begin{cases} 0, & \hat{v} < v_{\text{in}}, \\ P_{\text{rated}}\left(\frac{\hat{v}-v_{\text{in}}}{v_{\text{rated}}-v_{\text{in}}}\right)^3, & v_{\text{in}} \le \hat{v} \le v_{\text{rated}}, \\ P_{\text{rated}}, & v_{\text{rated}} < \hat{v} < v_{\text{out}}, \\ 0, & \hat{v} \ge v_{\text{out}}. \end{cases} \tag{28}$$

Let $\tilde{y}_{t+h}^{\text{raw}} = \hat{y}_{t+h}/P_{\text{rated}}$ and $\tilde{P}_{\text{env}}(\hat{v}_{t+h}) = P_{\text{env}}(\hat{v}_{t+h})/P_{\text{rated}}$. The forward projection is

$$\tilde{y}_{t+h}^{\text{phys}} = \text{clip}\Big(\tilde{y}_{t+h}^{\text{raw}},\ 0,\ \tilde{P}_{\text{env}}(\hat{v}_{t+h})\Big), \tag{29}$$

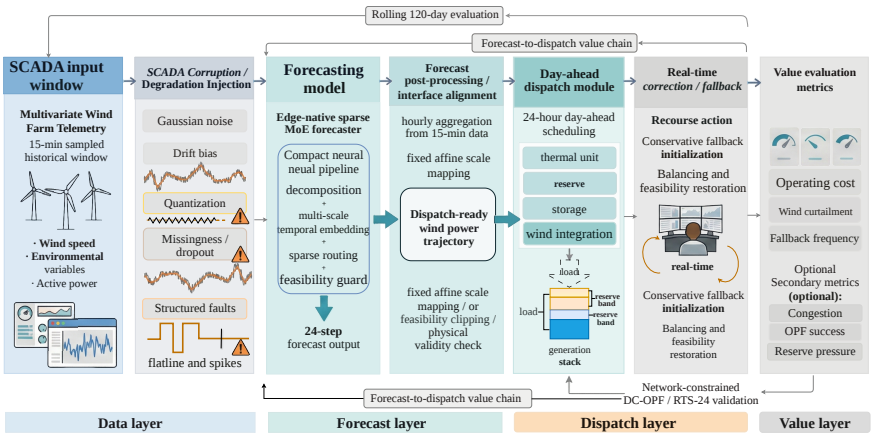

Figure 2: Dispatch-coupled evaluation pipeline from SCADA window to dispatch outcomes.

which is used here as a feasible-set projection in the same spirit as constrained screening/projection workflows in power-system optimization pipelines [7, 25, 27]. and the hinge-type physics loss is

$$\mathcal{L}_{\text{physics}} = \frac{1}{H}\sum_{h=1}^{H} w_h \Big[\max(0, -\tilde{y}^{\text{raw}}_{t+h})^2 + \max(0, \tilde{y}^{\text{raw}}_{t+h} - 1)^2 + \max(0, \tilde{y}^{\text{raw}}_{t+h} - \tilde{P}_{\text{env}}(\hat{v}_{t+h}))^2 \Big], \tag{30}$$

where $w_h = \exp(-h/\tau_p)$. Combining Eqs. (12), (20), and (30), the final objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}}(\tilde{\mathbf{y}}^{\text{phys}}, \mathbf{y}) + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}. \tag{31}$$

For reproducibility, end-to-end procedures are summarized in Algorithm 1 (training with imperfection injection) and Algorithm 2 (edge inference with Jump Gate); Fig. 2 provides a compact workflow from SCADA measurements to dispatch outcomes.

Figure 2 fixes the end-to-end chain used for dispatch evaluation, with scale mapping and feasibility clipping applied before DA/RT optimization. All downstream dispatch indicators are produced within this same pipeline.

3.5. Conditional skipping for edge inference

The Jump Gate defines token-wise conditional skipping. The gate score is

$$I_t = \text{Sigmoid}(\mathbf{w}_g^\top \mathbf{e}_t), \tag{32}$$

Algorithm 1 Ultra-LSNT training workflow with SCADA imperfection injection.

Input: training samples $(\mathbf{X}, \mathbf{Y})$ with $\mathbf{X} \in \mathbb{R}^{L \times C}$, $\mathbf{Y} \in \mathbb{R}^{H \times 1}$
Hyperparameters: experts N, Top-K, router regularizer λ_r, physics weight λ_2, noise config σ_{cfg}, industrial scale α, Jump Gate threshold τ
$\sigma_{\text{eff}} \leftarrow \alpha \cdot \sigma_{\text{cfg}}$
$\tilde{\mathbf{X}} \leftarrow \text{InjectImperfections}(\mathbf{X}; \sigma_{\text{cfg}}, \sigma_{\text{eff}})$
    (Gaussian noise, drift, quantization; see Sec. 4.2 for definitions.)
$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\tilde{\mathbf{X}})$
$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$
for $b = 1$ **to** N_{blk} **do**
    $\mathbf{g} \leftarrow \text{Router}_b(\mathbf{Z})$ *(routing probabilities)*
    $\mathcal{E} \leftarrow \text{TopK}(\mathbf{g}, K)$ *(selected experts)*
    $\mathbf{Z} \leftarrow \text{AggregateExperts}_b(\mathbf{Z}, \mathcal{E})$
    $\mathbf{Z} \leftarrow \mathbf{Z} + \text{TrendInteract}_b(\mathbf{Z}, \mathbf{X}_{\text{trend}})$
end for
$\hat{\mathbf{Y}}^{\text{raw}} \leftarrow \text{ForecastHead}(\mathbf{Z})$
$\hat{\mathbf{Y}}^{\text{phys}} \leftarrow \text{PhysicsClip}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}})$
$\mathcal{L}_{\text{pred}} \leftarrow \text{MSE}(\hat{\mathbf{Y}}^{\text{phys}}, \mathbf{Y})$
$\mathcal{L}_{\text{aux}} \leftarrow \text{LoadBalance}(\mathbf{g})$
$\mathcal{L}_{\text{physics}} \leftarrow \text{PhysicsBarrier}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}})$
Update parameters to minimize $\mathcal{L}_{\text{pred}} + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}$

with hard-threshold transition

$$\mathbf{h}_t = \begin{cases} \mathbf{h}_{t-1}, & I_t < \tau, \\ \text{MoE}(\mathbf{e}_t), & \text{otherwise}, \end{cases} \tag{33}$$

where τ is the threshold parameter.

During training, the hard gate is relaxed as

$$\tilde{z}_t = \sigma\big((I_t - \tau)/s\big), \qquad \mathbf{h}_t = \tilde{z}_t \cdot \text{MoE}(\mathbf{e}_t) + (1 - \tilde{z}_t) \cdot \mathbf{h}_{t-1},$$

where $s > 0$ is temperature.

Algorithm 2 Edge inference workflow with Jump Gate conditional skipping.

Input: window $\mathbf{X} \in \mathbb{R}^{L \times C}$
$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\mathbf{X})$
$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$
for $b = 1$ **to** N_{blk} **do**
  if $\text{JumpGate}_b(\mathbf{Z}; \tau) = \text{SKIP}$ **then**
    continue
  end if
  Route experts and aggregate as in Algorithm 1
end for
$\hat{\mathbf{Y}}^{\text{raw}} \leftarrow \text{ForecastHead}(\mathbf{Z})$
$\hat{\mathbf{Y}}^{\text{phys}} \leftarrow \text{PhysicsClip}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}})$
Output: $\hat{\mathbf{Y}}^{\text{phys}}$

Ultra-LSNT-Lite is defined by reducing sparse-module capacity with Top-1 routing:

$$(N, K, \#\text{MoE blocks}, d) \;\rightarrow\; (2, 1, 1, 96), \tag{34}$$

where d is the hidden dimension.

Complexity follows directly. Let d denote hidden dimension. A dense N-expert computation scales as $O(N \cdot L \cdot d^2)$, whereas Top-K routing scales as $O(K \cdot L \cdot d^2)$ with $K \ll N$. If Jump Gate is enabled, the effective token count is multiplied by the non-skip ratio.

4. Experimental setup and robustness protocol

This section summarizes the shared evaluation pipeline: data preparation, benchmark selection, robustness protocol, and dispatch-coupled validation.

4.1. Data, preprocessing, and tasks

The primary case study is ultra-short-term wind-power forecasting using SCADA measurements from a utility-scale wind farm in Xinjiang, China. The setup is intended to reflect operational conditions with sensor uncertainty and non-stationary weather. Inputs include wind and environmental variables, and the target is aggregated active power.

Table 1: Dataset characteristics and SCADA degradation protocol.

Item	Specification
Panel A	
Multi-domain dataset spatiotemporal characteristics	
Wind (CN) SCADA (primary)	15-min sampling, 121,528 records, train/test chronology 80%/20% (97,222/24,306), train range 2021-04-02 00:00 to 2022-03-12 15:45, test range 2022-03-12 16:00 to 2022-06-30 23:45, 8 input channels, target active power in kW.
Wind (US) transfer	5-min sampling, 105,120 records, full-year range 2012-01-01 00:00 to 2012-12-31 23:55, 10 input channels (time/covariates) with wind-power target in MW.
Air Quality transfer	Daily city-panel records, 588,644 samples after preprocessing, range 2015-01-01 to 2023-05-31, 7 pollutant/air-quality channels (AQI, PM2.5, PM10, SO_2, NO_2, CO, O_3).
GEFCom Load transfer	1-hour sampling, 38,070 records, range 2004-01-01 00:00:00 to 2008-06-30 05:00:00, univariate load target with calendar-aligned chronological split.
Unified task protocol	All datasets use chronological splitting, train-fitted z-score normalization, inverse scaling before metric reporting, and common QC (duplicate removal, missing-value handling, NaN/Inf sanity checks). Main forecasting setup uses lookback/horizon $(L, H) = (96, 24)$.
Panel B	
SCADA degradation and stress protocol (Wind CN main benchmark)	
Gaussian noise (inference-time)	Configured severity $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ with industrial scaling $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, $\alpha = 1.5$, giving effective tiers $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$.
Drift bias (random-walk)	Persistent bias injected as a channel-wise random walk with step scale $\sigma_{\text{eff}} \cdot \text{std}(x)$; this emulates calibration drift and low-frequency sensor offset accumulation.
Quantization noise	Deterministic rounding with step size $\Delta = \sigma_{\text{eff}} \cdot \text{std}(x)$, i.e., $x_t^{\text{quant}} = \Delta \cdot \text{round}(x_t/\Delta)$, to emulate low-precision SCADA/edge telemetry.
Structured SCADA faults	Flatline, spike/heavy-tail outlier, and correlated dropout with contiguous windows; severity levels $\{0.2, 0.6\}$, with level 0.6 used as the severe-certification slice in main robustness tables.
Missingness and interpolation	i.i.d. masking rate $p \in \{0.1, 0.3\}$ with paired `ffill`/linear interpolation (3 seeds, shared masks across models); burst/correlated missingness is represented by structured dropout faults.
Empirical severity anchor	Channel-wise diff-proxy calibration on Wind (CN) yields median 0.045, P90 0.091, max 0.121; therefore $\sigma_{\text{eff}} = 0.15$ is treated as empirical upper-band neighborhood, while $\{0.30, 0.45, 0.60\}$ are explicit stress envelopes.
Dispatch coupling protocol	For dispatch, 15-min Wind (CN) is aggregated to 1-hour resolution; 120-day rolling copper-plate and IEEE RTS-24 DC-OPF screening are both reported under fixed affine scale mapping and fixed feasibility clipping rules.

Table 1 fixes dataset scope, stress tiers ($\sigma_{\text{eff}} \in \{0.15, 0.30, 0.45, 0.60\}$), and dispatch aggregation (15-min to hourly) in one place. Training split, corruption protocol, and dispatch coupling are therefore anchored to one shared protocol instead of ad hoc per-model settings. Figure 3

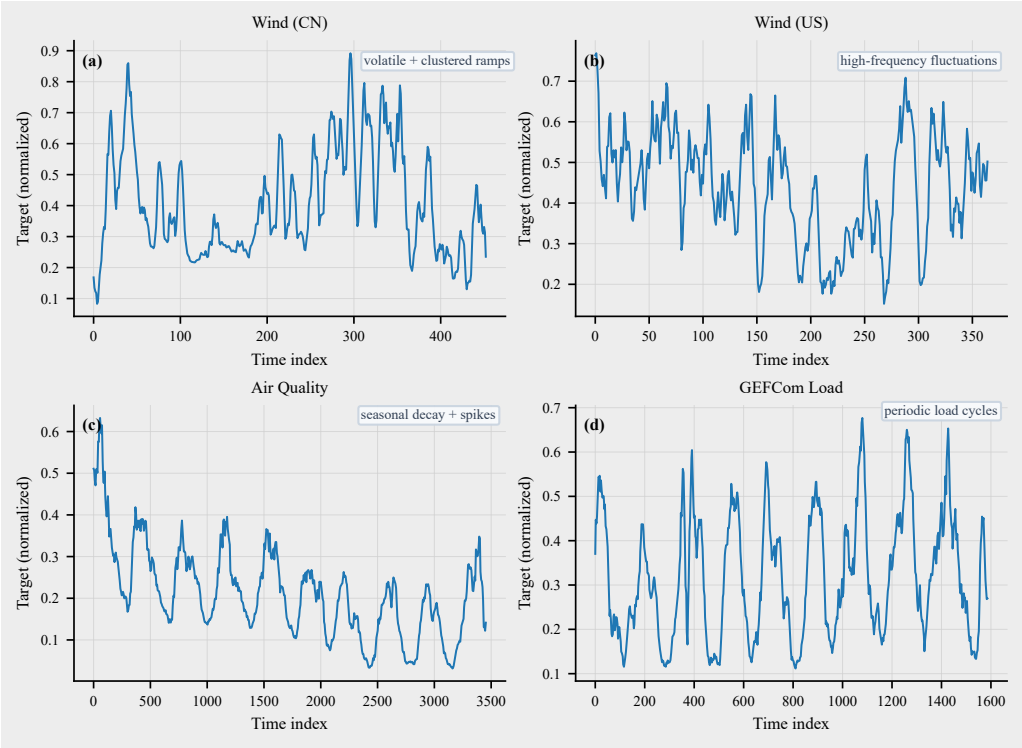

Figure 3: Representative raw-series excerpts for Wind (CN), Wind (US), Air Quality, and GEFCom Load. Series are min-max normalized within each dataset for visual comparability and plotted with dataset-specific excerpt length.

shows that the study covers heterogeneous temporal patterns rather than a single profile. To test transfer behavior, we include three additional tasks: *Wind (US)*, *Air Quality*, and *GEFCom Load*. All datasets are split chronologically to prevent leakage. Z-score normalization is fitted on the training split only and inverse scaling is applied before reporting physical-unit metrics. The same quality-control pipeline (duplicate removal, missing-value handling, NaN/Inf checks) is applied across all experiments.

4.2. Baselines, metrics, and robustness protocol

Main-text robustness evidence uses two aligned views. Table 2 Panel A is a clean-trained context panel with six models (DLinear, TimeMixer, iTransformer, Ultra-LSNT, LightGBM, Persistence) under identical test-time corruption. Panel B is a matched-control set (DLinear, TimeMixer, iTransformer, Ultra-LSNT) with clean-trained and fault-augmented counterparts (three seeds; four faults: Gaussian, drift, quantization, missingness). At the Gaussian severe

slice ($\sigma_{\text{eff}} = 0.60$), fault-augmented training changes mean R^2 from 0.2402 to 0.3471 (DLinear), 0.7452 to 0.8417 (TimeMixer), 0.8478 to 0.8580 (iTransformer), and 0.8135 to 0.8180 (Ultra-LSNT), indicating that robustness differences are driven by both architecture and training protocol. Long-tail baselines (including decomposition-optimization hybrids and constrained-budget metaheuristic probes with population=3, iterations=1, one-epoch objective/final fitting) are retained as diagnostic evidence in Appendix. Metadata for these diagnostic probes is archived in `results/supplementary_evidence/extended_metaheuristic_meta.json`. TSFMs such as Chronos and Moirai are excluded from primary comparative tables in this revision because their deployment profile does not match local controller constraints (memory, thermal limits, and latency).

LightGBM/Persistence protocol card: To keep non-deep baselines protocol-aligned with deep models, we use the same chronological split, window sizes ($L = 96$, $H = 24$), train-fitted z-score normalization with inverse scaling, and identical test-time corruption settings. Gaussian stress uses the same effective-noise rule $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$ with $\alpha = 1.5$. LightGBM is implemented as `MultiOutputRegressor(LGBMRegressor)` on flattened historical windows using only $x_{t-L+1:t}$ (no future covariates), with fixed hyperparameters (`n_estimators`=500, `learning_rate`=0.05, `random_state`=42, `n_jobs`=-1) to avoid asymmetric tuning budgets. Persistence is defined as repeating the last observed target over the full prediction horizon.

We evaluate point forecasting accuracy using R^2, RMSE, and MAE as the primary evidence, which reflect goodness-of-fit and absolute error in dataset-native physical units. We additionally define MAPE for completeness. For wind power series with frequent near-zero generation periods, ratio-based errors can be disproportionately inflated even with numerical safeguards, so MAPE is treated as a supplementary diagnostic rather than a headline metric.

$$R^2 = 1 - \frac{\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}{\sum_{t=1}^{T}(y_t - \bar{y})^2}. \tag{35}$$

$$\text{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}. \tag{36}$$

$$\text{MAE} = \frac{1}{T}\sum_{t=1}^{T}|y_t - \hat{y}_t|. \tag{37}$$

$$\text{MAPE} = \frac{100}{T}\sum_{t=1}^{T}\left|\frac{y_t - \hat{y}_t}{y_t + \epsilon}\right|. \tag{38}$$

Here $\bar{y}$ is the sample mean of y_t, and ϵ is a small constant used to avoid numerical instability when y_t approaches zero.

Average errors can hide tail risk and ramp-timing errors relevant to dispatch. We report tail-error, ramp, smoothness, and routing-stability indicators in addition to R^2/RMSE/MAE; for probabilistic variants, we also report PICP, PINAW, and CRPS. Robustness tests include continuous perturbations (Gaussian noise, drift, quantization) and structured faults (flatline, spikes, correlated dropouts), with Gaussian stress up to 60% of clean standard deviation and structured-fault severities $\{0.2, 0.6\}$. The main robustness table uses two views: Panel A (context panel) and Panel B (matched clean/fault-augmented controls). At test time, all models use identical corruption protocols with train-fitted inverse scaling. Full perturbation definitions are provided in Appendix D. We also report latency, model size, parameter count, and an analytical energy proxy under fixed profiling settings [35, 36].

4.3. Dispatch-coupling and reproducibility

For operational coupling, we formulate day-ahead (DA) scheduling on a 24-hour horizon with 1-hour resolution [37]. Forecast impact is summarized by three indicators: (i) operating cost, (ii) wind curtailment, and (iii) feasibility-restoration (fallback) frequency. Monetary outcomes are reported in CNY.

We also run a two-stage DA/RT DC-OPF supplement on IEEE RTS-24 and report total slack, congestion indicators, and OPF success ratio when available. This layer captures active-power congestion effects under linearized network physics [6].

To quantify forecast value in dispatch, we couple each model output to a simplified DA engine with one thermal unit, upward reserve procurement, and storage [28, 38]. Let $\hat{w}_t$ denote forecast wind power (kW), d_t demand (kW), and let dispatch variables be thermal output g_t (kW), reserve r_t (kW), curtailment u_t (kW), storage charge/discharge powers $p_{\mathrm{ch},t}$, $p_{\mathrm{dis},t}$ (kW), and state-of-charge s_t (kWh). The dispatch grid is hourly. We therefore aggregate Wind (CN) from 15-min to 1-hour resolution by averaging each 4 consecutive records (Table 1).

Ultra-LSNT is trained on this hourly series with $L = 96, H = 24$ and outputs $\hat{w}_{1:24}$ directly for dispatch input. With $\Delta t = 1$ h and horizon $T = 24$, every forecaster provides a 24-step daily vector $\hat{w}_{1:T}$. We evaluate a 120-day rolling window for Ultra-LSNT, iTransformer, TimeMixer, and DLinear. A day is tagged as `fallback_x0` when the optimizer needs conservative fallback initialization under unchanged algebraic constraints.

To align forecast magnitudes with dispatch operating range, we release forecast-scaling metadata and apply one global affine mapping before clipping:

$$\hat{w}_t^{\text{map}} = a\hat{w}_t + b. \tag{39}$$

$$a = \frac{40{,}000 - 20{,}000}{131{,}839 - 14{,}250} = 0.1701. \tag{40}$$

$$b = 17{,}576.3. \tag{41}$$

This mapping is fixed across models and is not tuned per model. Calibration uses a deterministic dispatch sandbox with $d_t = 50{,}000\,\text{kW}$ and $g_t \in [10{,}000, 80{,}000]\,\text{kW}$, where a wind corridor of $[20{,}000, 40{,}000]\,\text{kW}$ corresponds to 40–80% wind share and leaves thermal headroom for balancing and reserve actions.

Raw and mapped outcomes are both reported in Table 5. In raw-scale stress checks, all models stay in `fallback_x0` for all 120 days, with optimized ratio 0. After fixed mapping, optimized ratios become non-zero and the resulting metrics are finite across models. The corresponding rolling-dispatch summaries are archived in the supplementary evidence bundle.

To keep operating bounds consistent across models in rolling runs, we apply deterministic feasibility mapping to each daily forecast vector:

$$\tilde{w}_t = \text{clip}(s\hat{w}_t, 0, w_{\max}), \tag{42}$$

$$s = \min\left(1, \frac{w_{\max}}{\max_t \hat{w}_t + \epsilon}\right), \tag{43}$$

where $w_{\max} = \max(w_{\text{train}})$ is computed on the hourly training split and ϵ is a small constant. Results under this mapping are reported in Table 5, and both raw/mapped rolling summaries are included in supplementary evidence.

All monetary coefficients are in CNY as implemented in `evaluate_cost`. Because absolute totals depend on price coefficients and reference year, we report both absolute values and relative reductions against fixed baselines. With $\Delta t = 1$ h, summed kW schedules correspond to kWh energy over the horizon. We compute dispatch metrics against realized wind output on the same test day for all models (see `compare_dispatch_metrics.py` and `wind_dispatch_model.py`).

We include one UQ-aware reserve variant where $\sigma_{w,t}$ scales with routing entropy and expert disagreement from the MoE forward pass. For comparability, we also report the fixed-proportional reserve rule $\sigma_{w,t} = 0.2|\hat{w}_t|$.

Note on network constraints. The dispatch model above is single-node (copper-plate) and isolates how forecast errors propagate to reserve, curtailment, and operating cost. We additionally run network-constrained DC-OPF on IEEE RTS-24 using PYPOWER (`rundcopf`) with line thermal limits and archived wind-bus mapping metadata (`results/supplementary_evidence/dispatch_network_ieee24_dcopf_wmax260_meta.json`). DC-feasible schedules are not guaranteed to be AC-feasible.

All experiments run in one pipeline with fixed preprocessing, chronological splitting, and consistent metric computation. Full environment/training settings are reported in Appendix C. Source code is released at `https://github.com/b1ue13e/Ultra-LSNT` (MIT), and Wind (CN) data plus preprocessing/evaluation scripts are released for end-to-end reproduction.

5. Results

This section reports robustness, transfer, hardware/safety, dispatch, and (K, τ, λ_2) sensitivity results under the same evaluation pipeline.

5.1. Predictive robustness and transfer under SCADA corruption

Table 2 reports robustness on Wind (CN) through a context panel and a matched-control panel. In Panel A, DLinear is strongest on clean data ($R^2 = 0.9379$) but drops to $R^2 = -1.5790$ at $\sigma_{\text{eff}} = 0.60$, while Ultra-LSNT remains positive at $R^2 = 0.7263$. Panel B compares matched training protocols at the same severe slice and shows consistent gains from fault augmentation (for example, 0.2402 to 0.3471 for DLinear and 0.7452 to 0.8417 for TimeMixer).

Table 3 shows domain-dependent winners under one chronological protocol: Ultra-LSNT leads on Air Quality, Mamba on Wind (US), and SVR on GEFCom Load. No single model dominates all three tasks.

Figure 4 shows robustness trajectories under Gaussian, drift, and quantization corruption on Wind (CN), with anchors at $\sigma_{\text{eff}} = 0.15$ and 0.60. The full-σ sweep complements Table 2 by showing how performance gaps evolve across severity levels.

5.2. Hardware profiling and safety guardrails

Table 4 compares deployability and safety under protocol P1 (no-guardrail, clip-only, FG-MoE). FG-MoE reduces over-rated violations from 0.0550% to 0.0092% while preserving

Table 2: Protocol-consistent robustness evidence on Wind (CN).

Panel A

Clean-trained context panel (6 models)

Model	**Clean R^2**	R^2@0.60	ΔR^2	**Rel. drop (%)**
DLinear	0.9379	−1.5790	2.5170	268.36
TimeMixer	0.8886	0.8337	0.0549	6.18
iTransformer	0.9012	0.8717	0.0294	3.26
Ultra-LSNT	0.8728	0.7263	0.1465	16.78
LightGBM	0.9069	0.6156	0.2914	32.13
Persistence	0.9175	0.8612	0.0563	6.14

Panel B

Matched-control severe-slice panel

Model	**Clean-trained** R^2@0.60	**Fault-augmented** R^2@0.60	**Gain**
DLinear	0.2402	0.3471	+0.1069
TimeMixer	0.7452	0.8417	+0.0965
iTransformer	0.8478	0.8580	+0.0102
Ultra-LSNT	0.8135	0.8180	+0.0045

Note: Panel A provides a six-model context view at clean and severe Gaussian slices. Panel B provides matched clean-trained and fault-augmented controls for the four deep models. In Panel A, $\sigma_{\text{eff}} = 0.60$ is a representative strong-noise slice, while the full σ sweep is reported in Figure 4 (including LightGBM and Persistence). Panel A is reported with R^2 because it is the currently archived common metric across all six models; supplementary scale-error evidence (RMSE/MAE) in Table G.8 is restricted to the archived subset (matched-control four models plus Persistence) and does not include LightGBM. Sources: `results/robust_gaussian_windcn.csv`, `experiment_results_20260302/01_fairness_closure/fairness_closure_fullfault_summary.csv`, and manifest index `results/supplementary_evidence/main4_matrix_manifest.csv`.

clean/severe accuracy, whereas clip-only reaches 0% violations at a large accuracy cost. Panel B provides the main safety result, and Panel A provides deployment context with direct and backfilled ARM measurements.

5.3. Dispatch results under mapped comparison

Table 5 reports dispatch outcomes before and after mapping. Raw-scale runs are fallback-only for all models, while mapped runs produce finite metrics. Under mapped interfaces, ranking changes across affine, isotonic, and quantile transforms (mean Spearman = 0.60, Top-1 consistency = 0); Figure 5 shows cumulative and representative-day effects, and Figure 6 visualizes

Table 3: Cross-domain transferability under a unified chronological protocol.

Model	**Wind (US)** R^2	**Air Quality** R^2	**GEFCom Load** R^2
Ultra-LSNT	0.7878	**0.4861**	0.7554
DLinear	0.8779	0.3433	0.7370
iTransformer	0.8563	0.4590	0.7467
TimeMixer	0.8023	0.3869	0.7201
ARIMA (logged)	0.6971	0.2008	−0.4031
SVR (logged)	0.5796	0.1716	0.8226
SSA-ELM	0.8075	0.3152	0.7662
Mamba (logged)	0.8866	0.3688	−0.0083

Note: Values from `results/supplementary_evidence/multi_domain_transfer_matrix_all_models.csv`.

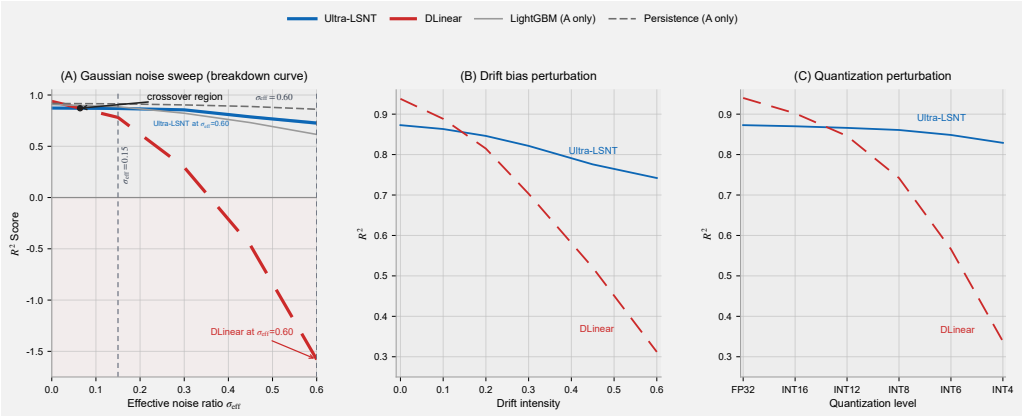

Figure 4: Robustness summary under Gaussian noise, drift bias, and quantization perturbations on Wind (CN). Panel A reports the full Gaussian sweep ($\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$) for Ultra-LSNT, DLinear, LightGBM, and Persistence, with $\sigma_{\text{eff}} = 0.15$ and $\sigma_{\text{eff}} = 0.60$ anchors. Panels B–C report drift and quantization trajectories for Ultra-LSNT and DLinear under the same protocol.

the rank-flip mechanism.

Panels B and C report L1 copper-plate and L2 RTS-24 DC-OPF outcomes from the same mapped forecasts. In L2, an hour is counted as successful only when both DA and RT OPF return `success`; under this criterion, curtailment gains can coexist with limited direct cost transfer under network constraints.

Table 4: Hardware profile and safety guardrail results under protocol P1.

Panel A

Batch-1 edge profile (CPU/ARM)

Model	CPU Latency (ms)	ARM Latency (ms)	CPU Throughput (Hz)	Memory (MiB)
Ultra-LSNT (full)	3.714	8.795	269.3	22.2830
Ultra-LSNT-Lite (opt.)	1.171	5.007	854.0	1.3327
DLinear	0.265	0.102	3767.6	0.0178
Mamba (logged) (bf)	0.861	0.861	1161.1	0.1468
VMD-PSO-LSTM (bf)	10.297	10.297	97.1	0.2468
CEEMDAN-WOA-GRU (bf)	11.082	11.082	90.2	0.2130
ARIMA (logged) (bf)	2.183	2.183	458.0	0.0000
SVR (logged) (bf)	2.750	2.750	363.6	0.5410
SSA-ELM (bf)	0.024	0.024	42409.6	0.8107

Panel B

Safety guardrail effect

Setting	Clean R^2	R^2@0.60	Over-rated violation (%)
Ultra-LSNT (no guardrail)	0.8727	0.8273	0.0550
Ultra-LSNT + clip-only (posthoc)	0.4319	0.3120	0.0000
FG-MoE (full-trained)	0.8747	0.8286	0.0092

Note: Panel A CPU values are from `results/supplementary_evidence/main4_matrix3_hardware_safety_joint.csv`; Panel A ARM values are from `results/supplementary_evidence/latency_arm_full_backfill.csv`; Panel B three-way guardrail values are from `results/supplementary_evidence/main4_matrix3_hardware_safety_joint.csv` (source audit: `experiment_results_20260302/04_guardrail_physics/physics_closure.csv`, protocol P1). Unmarked rows correspond to direct ARM logs in this evidence pack; rows tagged "(bf)" are backfilled proxy entries produced by `backfill_table4_arm_latency.py` from logged latency CSVs.

5.4. Hyperparameter Pareto frontier

Table 6 quantifies the accuracy-safety Pareto in (K, τ, λ_2) space under protocol P2 (joint-sensitivity SOA scan with router overrides, λ_2 post-guard correction, and partial-batch evaluation). In this setting, low λ_2 keeps severe-noise R^2 near 0.93 with non-zero over-rated risk, while $\lambda_2 = 1.0$ removes violations but sharply reduces severe-slice accuracy. Figure 7 shows the same trade-off at the anchor setting $(N, K, \tau) = (8, 2, 0.5)$ and is used to select operating points within P2.

Table 5: Dispatch outcomes under 120-day rolling and RTS-24 DC-OPF evaluation.

Panel A

Scale-mapping comparability gate

Model	Raw opt.	Raw FB (%)	Mapped opt.	Mapped FB (%)
DLinear	0.0000	100.0	0.3750	62.5
TimeMixer	0.0000	100.0	0.7833	21.7
iTransformer	0.0000	100.0	0.8333	16.7
Ultra-LSNT	0.0000	100.0	0.7583	24.2

Panel B

L1 mapped-protocol outcomes (copper-plate)

Model	L1 Cost (M CNY)	L1 Curt. (M kWh)	L1 FB (%)
iTransformer	32.16	15.22	16.7
TimeMixer	34.90	19.60	21.7
Ultra-LSNT	35.97	22.09	24.2
DLinear (base)	54.47	55.71	62.5

Panel C

L2 mapped-protocol outcomes (RTS-24 DC-OPF)

Model	L2 Curt. (MWh)	L2 Succ. (%)	L2 Gain (%)
iTransformer	138,093.39	100.00	+0.94
TimeMixer	138,093.39	100.00	+0.94
Ultra-LSNT	138,093.39	100.00	+0.94
DLinear (base)	139,403.69	100.00	+0.00

Note: Panel A from `results/supplementary_evidence/dispatch_raw_vs_mapped_summary.csv`; Panel B from `results/supplementary_evidence/main4_matrix4_dispatch_value.csv`; Panel C from `results/supplementary_evidence/main4_matrix4_dispatch_value.csv`, cross-checked against `experiment_results_20260302/02_dispatch_closure/dispatch_closure_mapping_decision_aggregate.csv` and `experiment_results_20260302/02_dispatch_closure/dispatch_closure_mapping_decision_ranking_stability.csv`.

6. Discussion and limitations

Protocol note: This paper uses one evidence hierarchy with four boundaries. First, mapped dispatch ranking is interface-sensitive (mean Spearman = 0.60, Top-1 consistency = 0). Second, P1 safety values in Table 4 and P2 SOA-proxy values in Table 6 come from different protocols and are not compared as same-name metrics. Third, L1 copper-plate and L2 RTS-24 DC-OPF

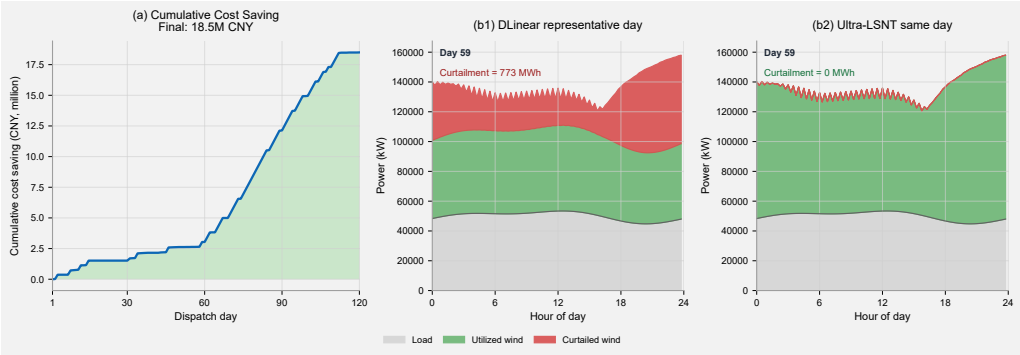

Figure 5: Dispatch value evidence under the fixed mapped protocol. Panel (a) shows 120-day cumulative cost saving against DLinear. Panels (b1) and (b2) show the representative day for DLinear and Ultra-LSNT, respectively.

Table 6: Pareto operating points in (K, τ, λ_2) at $\sigma_{\text{eff}} = 0.60$ under protocol P2 (SOA proxy).

K	τ	λ_2	**Latency (ms)**	R^2@0.60	**Over-rated (SOA proxy, %)**
1	0.7	0.01	10.609	0.9301	19.1332
1	0.9	0.10	10.232	0.9257	18.0169
2	0.5	0.01	14.353	0.9294	19.0091
4	0.9	0.01	19.769	0.9299	18.9690
1	0.9	1.00	10.232	0.3831	0.0000
4	0.5	1.00	18.182	0.3913	0.0000

Note: Values sampled from `results/supplementary_evidence/joint_sensitivity_pareto_frontier.csv` (protocol P2: router override + λ_2 post-guard + partial-batch evaluation); full grid in `results/supplementary_evidence/joint_parameter_full_windcn.csv`. Comparability boundaries between P1 and P2 are summarized in the protocol note in Section 6.

are screening layers rather than operator certification. Fourth, DC-feasible schedules are not AC-feasible guarantees. Clean-data ranking and severe-corruption ranking diverge in the matched-control setting. Within the protocol above, Table 4 provides the primary safety evidence, Table 5 plus Figures 5–6 provide mapped-interface dispatch evidence, and Table 6 plus Figure 7 provide P2 operating diagnostics. For mechanism audit and directional statistical support, readers should consult the supplementary interpretability figures (Figures G.12–G.13) and DM significance matrix (Figure G.14); these validate inspectability and direction stability, not additional superiority claims.

Three limitations remain. First, dispatch ranking depends on the chosen mapping interface.

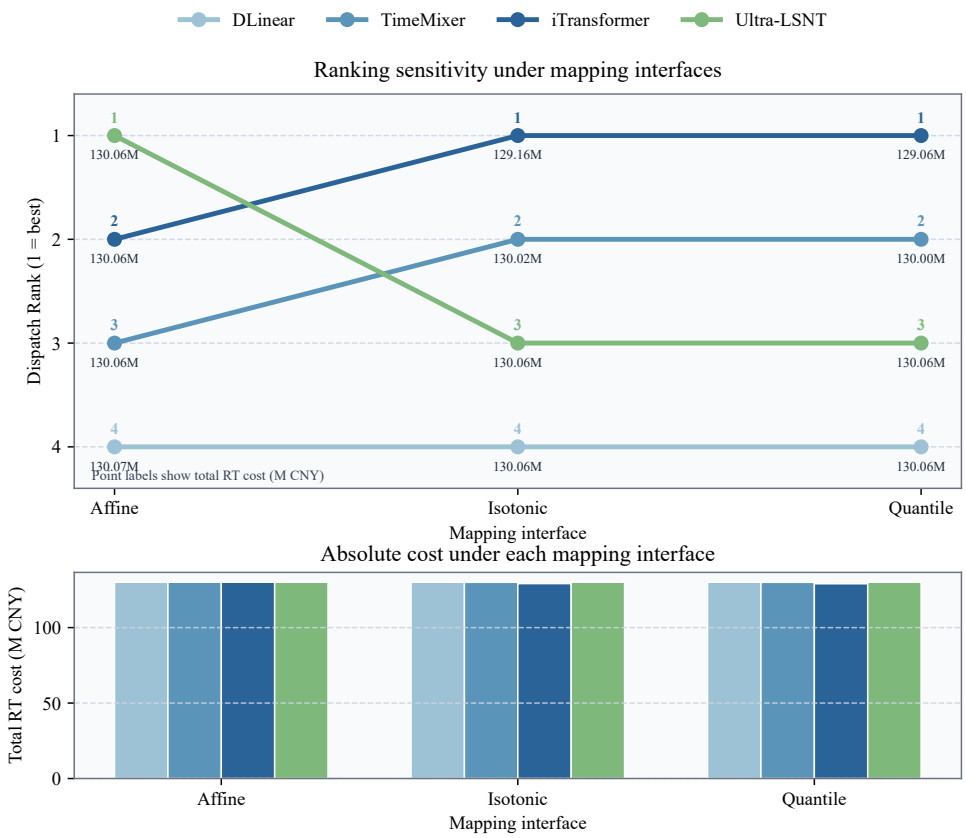

Figure 6: Ranking sensitivity under affine, isotonic, and quantile mappings. The upper panel reports rank by total RT cost, and the lower panel reports total RT cost. Summary statistics: mean Spearman rank correlation = 0.60, Top-1 consistency = 0.

Second, the current operational layer stops at copper-plate plus RTS-24 DC-OPF screening and does not certify AC feasibility. Third, latency profiling is still pre-HIL, and long-tail supplementary baselines have not yet received full fault-augmented matched controls.

7. Conclusion and outlook

This study shows that corrupted-input robustness and output-feasibility control can be improved together in one edge-oriented forecasting pipeline. On Wind (CN), Ultra-LSNT keeps positive severe-corruption skill in a regime where some high-clean baselines collapse, and FG-MoE strongly suppresses over-rated outputs.

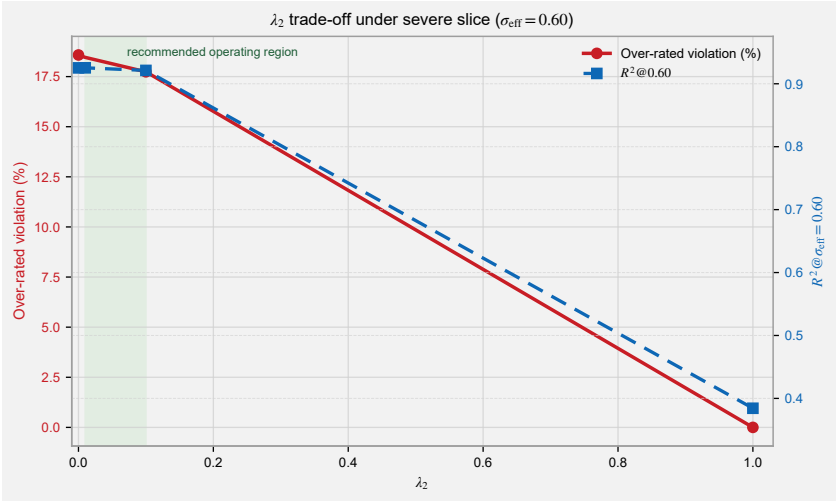

Figure 7: λ_2 safety-accuracy trade-off at anchor $(N, K, \tau) = (8, 2, 0.5)$ under the severe slice $\sigma_{\text{eff}} = 0.60$. The left axis reports over-rated violation (%), and the right axis reports R^2@0.60. The shaded band indicates the recommended operating region.

The current evidence does not support a single best model across all tasks and mapped interfaces. Strengths remain task-dependent, dispatch ranking changes when the mapping choice changes, and main comparative conclusions are restricted to the four-model matched-control set (DLinear, TimeMixer, iTransformer, Ultra-LSNT); long-tail and low-budget metaheuristic entries remain supplementary diagnostics.

The next key validation step is system-grade closure: AC/operator-grade dispatch evaluation with site-specific calibration, together with board-level HIL verification of latency, power, and thermal behavior.

CRediT authorship contribution statement

Junyu Li: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft, Visualization. **Juntao Du**: Supervision, Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Wind (CN): The Wind (CN) SCADA dataset used in this study is released publicly to support full reproducibility. The release includes the raw time series, the train/validation/test split indices, and the preprocessing scripts used to construct the learning task described in Table 1. The dataset package is distributed via the project repository (`https://github.com/b1ue13e/Ultra-LSNT`) and will be mirrored to a long-term archival service at acceptance.

Public benchmarks: The additional datasets used for cross-domain evaluation are publicly accessible: (i) Wind (US) from the NREL Wind Toolkit [39], (ii) Air Quality from the UCI repository [40], and (iii) GEFCom load from the Global Energy Forecasting Competition (GEFCom2012) [41]. We report the exact dataset identifiers, preprocessing scripts, and evaluation protocols in the released code.

In line with Elsevier's research data practices, we provide reproducibility assets (protocol definitions, supplementary evidence tables, and run scripts) in the project repository under `results/supplementary_evidence`.

Code: The implementation, training scripts, and evaluation protocols are available at `https://github.com/b1ue13e/Ultra-LSNT` (MIT license).

Appendix A. Nomenclature and abbreviations

Item	Description
Abbreviations	
MoE	Mixture-of-Experts
SCADA	Supervisory Control and Data Acquisition
TSFM	Time-series foundation model
SOTA	State-of-the-art
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
FG-MoE	Feasibility-Guarded MoE (main-text term in this manuscript)

Item	Description
UQ	Uncertainty Quantification
CRPS	Continuous Ranked Probability Score
AC-OPF	AC Optimal Power Flow
DC-OPF	DC Optimal Power Flow
HIL	Hardware-in-the-Loop
Symbols	
L	input length (time steps)
H	prediction horizon (time steps)
C	number of input variables/covariates
$\mathbf{X} \in \mathbb{R}^{L\times C}$	input sequence
$\hat{\mathbf{Y}} \in \mathbb{R}^{H\times 1}$	forecast output (wind power / target variable)
σ_{cfg}	configured perturbation ratio in robustness tests (dimensionless)
σ_{eff}	effective perturbation ratio, $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$
α	scaling factor used to map configured to effective noise severity ($\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, default $\alpha = 1.5$)
K	number of selected experts in Top-K routing
λ_r	load-balancing regularization coefficient for routing
τ	Jump Gate threshold for conditional skipping
R^2	coefficient of determination (scale-free)
RMSE/MAE	absolute errors in original physical units after inverse scaling
E_{inf}	estimated energy per sample inference (mJ)
B	batch size used in profiling (dimensionless)
N_{call}	number of forecasting calls per day
N_{turb}	number of turbines in a wind farm or fleet
c_f	fuel / purchase price coefficient in dispatch (CNY/kWh)
c_r	reserve price coefficient (CNY/kWh)
c_s	storage discharge cost coefficient (CNY/kWh)
c_{su}	start-up / shut-down cost coefficient (CNY)
$P_{\text{ch,max}}$	storage charging power limit (kW)
$P_{\text{dis,max}}$	storage discharging power limit (kW)
S_{max}	storage energy capacity limit (kWh)
R_{max}	thermal ramp-rate limit (kW per hour)
$\bar{P}_{\text{dev}}$	average device power during inference (W)
ρ	air density (kg/m^3), estimated from temperature/pressure
A	rotor swept area (m^2)
v	wind speed at hub height (m/s)
$C_p(\lambda_{\text{tsr}}, \beta)$	power coefficient (Betz-limited)
λ_{tsr}	tip-speed ratio (dimensionless)
β	pitch angle (deg)
P_{rated}	rated-power upper bound (kW)
$v_{\text{in}}, v_{\text{out}}$	cut-in / cut-out wind speeds (m/s)

Item	Description
w	median-smoothing window length in FG-MoE dynamic proxy ($w = 5$)
κ	clipping coefficient in robust dynamic proxy ($\kappa = 3$)
$\Delta v_t^{\mathrm{med}}$	median-smoothed wind-speed derivative used in FG-MoE
s_t^{MAD}	robust derivative scale estimated by MAD in FG-MoE
λ_2	plausibility-penalty coefficient in heuristic FG-MoE (dimensionless)

Appendix B. Dispatch model details

This appendix provides the deterministic dispatch algebra used by the screening workflow in Section 4.3. The formulation is included for reproducibility and serves as comparative screening infrastructure rather than AC-secure operational guidance.

With horizon $T = 24$ and $\Delta t = 1$ h, the objective is

$$\min \; C_{\mathrm{dispatch}} = C_{\mathrm{fuel}} + C_{\mathrm{reserve}} + C_{\mathrm{purchase}} + C_{\mathrm{storage}} + C_{\mathrm{startup}} + C_{\mathrm{shutdown}}, \tag{B.1}$$

$$C_{\mathrm{fuel}} = c_f \sum_{t=1}^{T} g_t \Delta t,$$

$$C_{\mathrm{reserve}} = c_r \sum_{t=1}^{T} r_t \Delta t,$$

$$C_{\mathrm{purchase}} = c_f \sum_{t=1}^{T} p_{\mathrm{ch},t} \Delta t,$$

$$C_{\mathrm{storage}} = c_s \sum_{t=1}^{T} p_{\mathrm{dis},t} \Delta t,$$

$$C_{\mathrm{startup}} = c_{\mathrm{su}} \sum_{t=2}^{T} \frac{\max(g_t - g_{t-1}, 0)}{R_{\max}},$$

$$C_{\mathrm{shutdown}} = c_{\mathrm{su}} \sum_{t=2}^{T} \frac{\max(g_{t-1} - g_t, 0)}{R_{\max}}.$$

Subject to

$$g_t + \hat{w}_t + p_{\mathrm{dis},t} - p_{\mathrm{ch},t} - u_t = d_t, \tag{B.2}$$

$$0 \le u_t \le \hat{w}_t, \quad \forall t,$$

$$g_{\min} \le g_t \le g_{\max}, \tag{B.3}$$

$$0 \le r_t, \quad g_t + r_t \le g_{\max},$$

$$|g_t - g_{t-1}| \le R_{\max},$$

$$0 \le p_{\mathrm{ch},t} \le P_{\mathrm{ch,max}}, \tag{B.4}$$

$$0 \le p_{\mathrm{dis},t} \le P_{\mathrm{dis,max}},$$

$$s_t = s_{t-1} + \eta_{\mathrm{ch}} p_{\mathrm{ch},t} \Delta t - \eta_{\mathrm{dis}}^{-1} p_{\mathrm{dis},t} \Delta t, \tag{B.5}$$

$$0 \le s_t \le S_{\max}.$$

and the fixed-proportional reserve protocol

$$\sigma_{w,t} = 0.2 \cdot |\hat{w}_t|, \; t = 1, \ldots, T. \tag{B.6}$$

which is a deterministic comparison rule and not UQ-aware dynamic reserve optimization.

Appendix C. Implementation and environment details

Appendix C.1. Implementation details and computational environment

All experiments are implemented in Python (v3.10) using PyTorch (v2.0+) with CUDA (v11.8) acceleration. The core scientific stack includes NumPy (v1.24+) and Pandas (v2.0+) for data handling, Matplotlib (v3.7+) and Seaborn (v0.12+) for visualization, and LightGBM (v4.0+) with Scikit-learn (v1.3+) for tree-based baselines. These versions are reported to ensure that both deep and non-deep baselines can be reproduced under consistent dependencies. The implementation is openly available at `https://github.com/b1ue13e/Ultra-LSNT` (MIT license), enabling inspection of model components (e.g., sparse routing, skipping gate) and exact reproduction of the reported protocols.

The main training is conducted on a workstation equipped with an NVIDIA GeForce RTX 4090 GPU, an Intel i9-13900K CPU, and 64 GB RAM for reproducible optimization and ablation sweeps. To complement workstation profiling, we report batch-1 inference latency on

commodity CPUs and an ARMv8 (aarch64) single-core platform (Huawei Kunpeng 920). The Kunpeng run is used as a server-class ARM reference for cross-ISA comparison rather than as an edge-device surrogate.

Implementation and training: We implement Ultra-LSNT in native PyTorch without third-party model dependencies. The sparse MoE core uses a custom `SparseMoELayer` with Top-K routing ($K = 2$ in the main model), and the dynamic skip path uses `ConsistentSkipGate` with `DualModePropagation`. To stabilize routing, the released configuration sets `aux_loss_coef` to $\lambda = 0.01$ and keeps the router z-loss at 0.01, following standard MoE load-balancing and router-stabilization practice [31, 32]; a controlled $\lambda \in \{0.01, 0.05\}$ robustness check is summarized in Table 4. The code mapping is explicit: MoE modules are defined in `ultra_lsnt_timeseries.py`, and training entry points are `train_ultra_lsnt_stable.py` and `train_ultra_lsnt_enhanced.py`. Unless otherwise stated, we train with $L = 96$, $H = 24$, chronological splits, train-fitted z-score normalization, AdamW $(\beta_1, \beta_2) = (0.9, 0.999)$, weight decay 0.01, cosine annealing with warm-up, initial learning rate 10^{-3}, and batch size 128; typical runs last 10–50 epochs depending on convergence.

Deployment profiling and reproducibility: For deployment-oriented profiling, we enable `torch.compile` (max-autotune), memory reuse, and an optimized Lite fast path when applicable. We report these switches explicitly because efficiency claims depend on runtime configuration; in particular, compile/AMP is enabled only for selected Lite rows and is reported as a reproducible deployment recipe rather than a fully normalized cross-model systems bake-off. We profile inference under fixed batch-1 settings for x86/ARM latency screening (Table 4). To keep experiments reproducible, we fix random seeds (seed=42), enable deterministic CUDA settings when possible, run in isolated environments, and archive logs in dedicated directories (e.g., `logs/`, `logs_complete/`) with separated raw/preprocessed data paths.

Appendix D. Robustness protocol details

Appendix D.1. Perturbation definitions and comparison setup

Additive Gaussian noise: sensor inaccuracy stress test. Additive Gaussian corruption approximates short-lived measurement noise, such as electromagnetic interference and turbulent sensing jitter in SCADA channels. Formally, for a clean input series x, we generate a corrupted

series

$$x_{\text{noisy}} = x + \sigma_{\text{eff}}\,\text{std}(x) \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 1), \tag{D.1}$$

where $\text{std}(x)$ is the standard deviation of the clean signal and σ_{eff} is the effective noise ratio. In our released noise generator, we use an industrial scaling factor $\alpha = 1.5$. To strengthen empirical grounding, we additionally compute a channel-wise proxy noise ratio on Wind (CN) core channels (windspeed, winddirection, temperature, humidity, pressure, power):

$$r_c = \frac{\text{std}(\Delta x_c)}{\sqrt{2}\,\text{std}(x_c)}, \tag{D.2}$$

where $\Delta x_c = x_{t,c} - x_{t-1,c}$. On this dataset, the proxy statistics are median 0.045, P90 0.091, and max 0.121. Detailed CSV/JSON calibration artifacts are archived in the supplementary-evidence bundle.

Based on this anchor, we treat $\sigma_{\text{eff}} = 0.15$ as an empirical upper-band neighborhood, while $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ are explicit stress-envelope settings for robustness certification under abnormal SCADA conditions rather than nominal sensor-error claims. Throughout the manuscript, severity is reported by the *effective* ratio $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, so the injected noise has standard deviation $\sigma_{\text{eff}} \cdot \text{std}(x)$. In configuration files and scripts, the user-facing parameter is σ_{cfg}; we therefore report both σ_{cfg} and σ_{eff} when needed. Unless otherwise stated, the Gaussian sweep uses $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ (equivalently $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$).

Structured SCADA faults: flatlining, spikes, and correlated dropouts. Additive Gaussian noise provides controlled severity scaling, but real SCADA failures usually exhibit temporal persistence and cross-channel dependence. We synthesize three structured fault families on top of clean streams: flatlining (sensor-freeze windows generated by dwell-time processes such as geometric or semi-Markov sampling), spikes/heavy-tailed outliers (sparse impulsive corruption with amplitudes from Laplace or Student-t distributions), and correlated multi-channel dropouts (common-cause masking events such as DAQ resets that affect multiple variables jointly for contiguous windows). We evaluate these faults on Wind (CN) at severities 0.2 and 0.6; the severe slice is summarized in Table 2, while full multi-severity rows are moved to supplementary material.

To complement deep-model comparisons, we also run focused non-deep context baselines (LightGBM and Persistence) over configured noise $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ (effective $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ with $\alpha = 1.5$), so severe-slice behavior is inspected with both

feature-engineered and naive-history references.

Special perturbations: drift and quantization noise. Beyond additive Gaussian noise, we evaluate two perturbation families that are common in industrial sensing and edge pipelines: *drift noise* and *quantization noise*. Drift noise approximates systematic bias that accumulates over time due to calibration drift, sensor aging, or persistent environmental interference. Quantization noise approximates rounding and discretization errors introduced by low-precision telemetry, bandwidth-constrained transmission, or edge-side INT8/FP16 processing. To keep perturbation severity comparable, for both perturbations we report severity by the same effective ratio σ_{eff} used in the Gaussian protocol. Unless otherwise stated, we sweep

$$\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}, \tag{D.3}$$

which corresponds to configured $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ when $\alpha = 1.5$.

Drift (random-walk bias): Given a clean channel $x_{1:L}$, we draw i.i.d. $\varepsilon_t \sim \mathcal{N}(0, 1)$ and define a persistent bias process

$$b_1 = 0. \tag{D.4}$$

$$b_t = b_{t-1} + \sigma_{\text{eff}} \cdot \text{std}(x) \cdot \varepsilon_t, \; t = 2, \ldots, L. \tag{D.5}$$

then corrupt the input as $x_t^{\text{drift}} = x_t + b_t$. The process is applied independently to each feature channel and each test sample. We reset $b_1 = 0$ per sample for paired reproducibility, and use the same random seed across methods at a given σ_{eff}; long contiguous fault persistence is covered separately by the structured-fault protocol.

Quantization (step-size rounding): We implement quantization as deterministic rounding to a channel-wise step size proportional to the clean-signal standard deviation:

$$\Delta = \sigma_{\text{eff}} \cdot \text{std}(x). \tag{D.6}$$

$$x_t^{\text{quant}} = \Delta \cdot \text{round}\left(\frac{x_t}{\Delta}\right). \tag{D.7}$$

where round(·) is round-to-nearest (half away from zero). When $\sigma_{\text{eff}} = 0$, no quantization is applied. This protocol yields a reproducible, severity-controlled discretization without requiring dataset-specific bit-range tuning. We report the corresponding robustness outcomes in Section 5.1 and interpret their operational implications in the Discussion, with emphasis on reliability under realistic sensing constraints and energy-aware (Green AI) deployment.

Missingness and interpolation artifacts: SCADA telemetry frequently contains dropouts due to communication loss or sensor resets, and downstream pipelines often apply simple imputation such as forward-fill or linear interpolation. To emulate this setting, we sample a binary mask $m_{t,c} \sim \text{Bernoulli}(1-p)$ over time steps t and channels c and define $\bar{x}_{t,c} = m_{t,c}x_{t,c} + (1-m_{t,c})\text{NaN}$. We then apply an imputation operator $\text{Imp}(\cdot)$ to obtain the corrupted series

$$x^{\text{miss}} = \text{Imp}(\bar{x}; \texttt{ffill/linear}), \tag{D.8}$$

where `ffill` denotes forward filling and `linear` denotes piecewise-linear interpolation along the time axis. In the reported experiments, we use i.i.d. masking (no burst blocks) as a controlled imputation stress test, sweep $p \in \{0.1, 0.3\}$, and repeat each configuration for three random seeds (0,1,2). For a fixed seed, the same missingness mask and imputation configuration are applied across models to preserve a paired comparison. Contiguous and correlated missingness is represented by the structured-fault dropout protocol. Results are provided in supplementary material.

Appendix E. Additional deployment and sensitivity figures

Figures E.8–E.11 address four supplementary questions: where deployment bottlenecks emerge under severe corruption, how efficiency axes co-vary, how Jump Gate threshold shifts the accuracy-latency trade-off, and how the structural N-K-τ landscape behaves at fixed λ_2. Figure E.8 isolates corruption-level deployment bottlenecks, Figure E.9 visualizes accuracy-efficiency coupling, Figure E.10 tracks threshold-driven accuracy-latency shifts, and Figure E.11 summarizes structural Pareto geometry. For Figure E.8, severe-slice R^2 follows the matched-control track and is paired with Panel-A CPU latency from Table 4 as supplementary deployment context.

Appendix F. Claim scope and practical boundaries

Dispatch mapping dependence: Dispatch ranking depends on the selected mapping interface (affine/isotonic/quantile) with feasibility clipping; raw-scale logs are released as sensitivity evidence.

Hardware status: Cross-ISA latency profiling is pre-HIL, and board-level validation on target PLC/RTU hardware remains required.

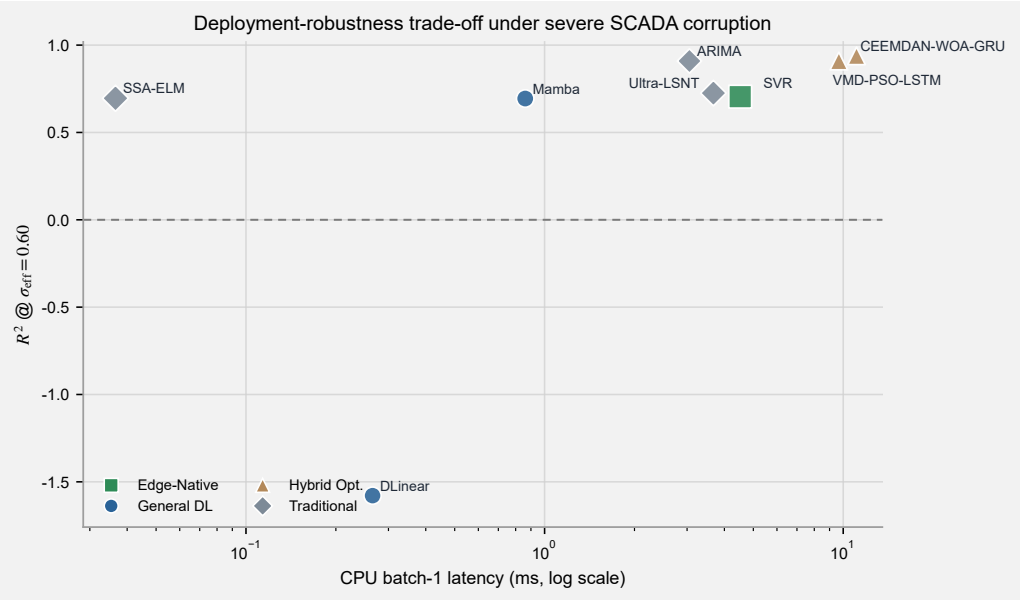

Figure E.8: Deployment-robustness trade-off under severe SCADA corruption ($\sigma_{\text{eff}} = 0.60$). CPU batch-1 latency is shown on the x-axis (log scale), and R^2@0.60 is shown on the y-axis. Point size encodes model size, and colors/markers indicate model family.

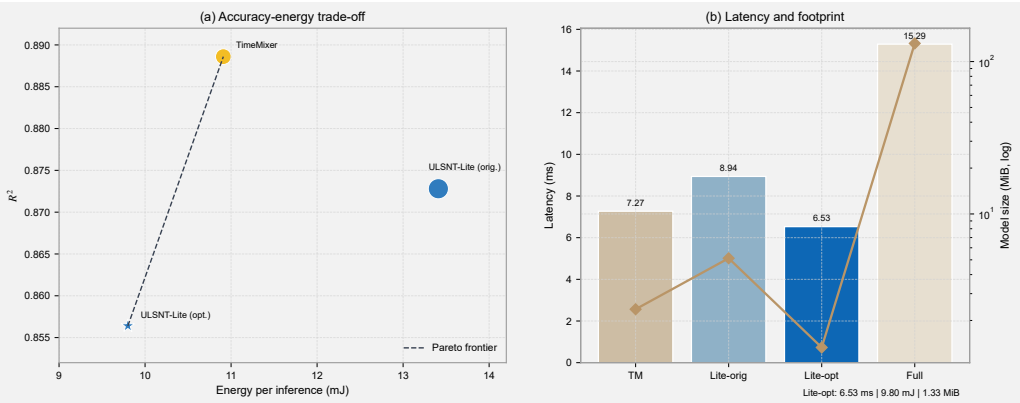

Figure E.9: Deployment-efficiency comparison across Ultra-LSNT variants and principal baselines. Energy, latency, and memory are reported jointly as edge-suitability indicators.

Matched-control status: This revision adds matched clean/fault-augmented controls for DLinear, TimeMixer, iTransformer, and Ultra-LSNT (three seeds; four faults), but full coverage for long-tail supplementary baselines remains future work.

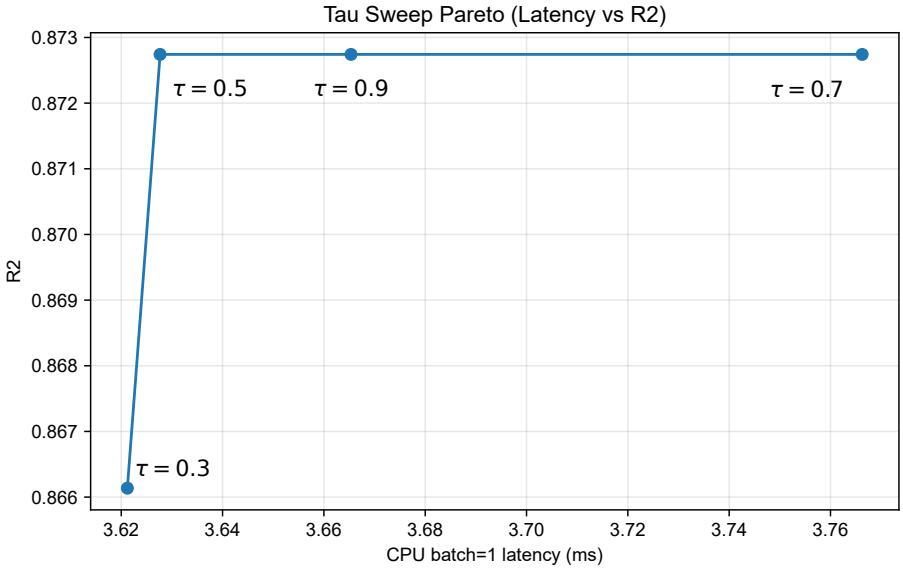

Figure E.10: Jump Gate threshold sweep in supplementary view. The threshold-latency-accuracy trajectory should be read jointly with the archived skip-ratio logs.

Appendix G. Supplementary tables: ablations, interpretability, and significance

Appendix G.1. Evidence-file mapping for primary comparative numbers

To keep the audit trail explicit, primary comparative numbers in this manuscript are mapped to archived result files rather than ad hoc extraction. Table 2 Panel A maps to `results/robust_gaussian_windcn.csv`; Table 2 Panel B maps to `experiment_results_20260302/01_fairness_closure/fairness_closure_fullfault_summary.csv`; dispatch and hardware primary rows map to their matrix CSVs listed in `results/supplementary_evidence/main4_matrix_manifest.csv`.

Appendix G.2. Scale-error supplementary table for robustness audit

Table G.8 provides archived RMSE/MAE severe-slice evidence for the currently available subset (matched-control four models plus Persistence) used to complement the R^2-only six-model context panel in Table 2. LightGBM scale-error values at $\sigma_{\text{eff}} = 0.60$ are not archived in the current bundle. The table is a transparency supplement, not a replacement for the main comparative track.

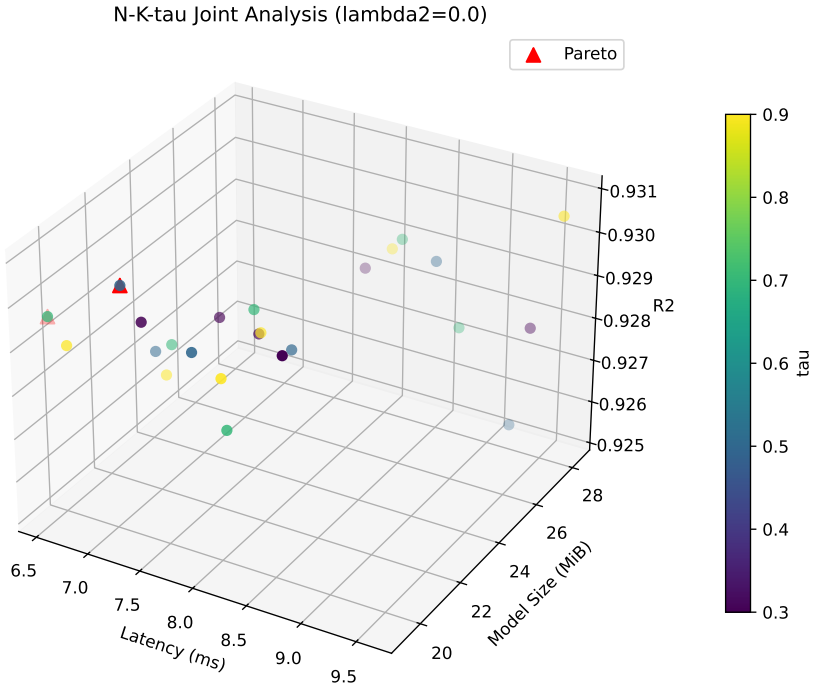

Figure E.11: 3D structural map (N-K-τ) at $\lambda_2 = 0$ with Pareto points highlighted (Wind CN, $\sigma_{\text{eff}} = 0.60$). Main-text decision evidence is reported in Table 6 and Figure 7.

Appendix G.3. Full structured SCADA fault robustness table

Complete multi-severity and multi-metric structured-fault results for Wind (CN) are reported in supplementary material and linked to `results/supplementary_evidence/structured_scada_fault_robustness_windcn.csv`.

Appendix G.4. Diagnostic long-tail baseline matrix (diagnostic-only)

For transparency, we retain broad baseline rows, including traditional methods, hybrid methods, and constrained-budget metaheuristics, in the supplementary evidence bundle. These rows are diagnostic only and are excluded from the matched-control main comparisons.

Appendix G.5. Ablation results

All ablations are conducted under a fixed training budget and identical evaluation protocol; we interpret them primarily through relative ranking and directional sensitivity, rather than expecting the absolute R^2 to exactly match the full benchmark runs.

Table G.8: Supplementary scale-error evidence at clean and $\sigma_{\text{eff}} = 0.60$ for the archived subset (Wind CN).

Model	**Mode**	**RMSE (clean)**	**MAE (clean)**	**RMSE@0.60**	**MAE@0.60**
DLinear	clean-trained	6065.98	3865.18	22190.19	15183.13
TimeMixer	clean-trained	9081.04	6789.63	12848.91	9674.21
iTransformer	clean-trained	7743.06	6029.31	9933.03	7852.08
Ultra-LSNT	clean-trained	9462.65	7360.92	10992.42	8705.76
DLinear	fault-augmented	6328.26	4181.08	20569.46	14105.63
TimeMixer	fault-augmented	8969.46	7067.89	10129.43	8034.08
iTransformer	fault-augmented	8779.48	6975.59	9593.71	7700.04
Ultra-LSNT	fault-augmented	9925.64	7858.85	10859.03	8675.17
Persistence	clean-trained	7313.94	4760.16	9485.55	6783.51
LightGBM	clean-trained	N/A	N/A	N/A	N/A

Note: Matched-control rows are extracted from `experiment_results_20260302/01_fairness_closure/fairness_closure_fullfault_summary.csv` (`test_fault=clean/gaussian`, $\sigma_{\text{eff}} = 0.60$). Persistence rows are from `results/supplementary_evidence/persistence_gaussian_windcn.csv`. LightGBM scale metrics at $\sigma_{\text{eff}} = 0.60$ are marked N/A because they are not archived in the current robustness bundle. On the audited subset with available scale metrics, severe-slice directional trends are consistent with the corresponding R^2 changes; this is an empirical subset observation, not a universal metric-equivalence claim.

Supplementary ablation tables report kernel-size sensitivity from the archived ablation CSV. On Wind (CN), $k = 3$ and $k = 7$ are close, while $k = 15$ is weaker. This supports using compact receptive fields for turbulence-dominant SCADA dynamics. Router-regularization robustness is reported in Table 4 using the two archived λ checkpoints (0.01 and 0.05).

Appendix G.6. XAI analysis: routing diagnostics for operational audit

Figures G.12 and G.13 visualize sparse-routing statistics at complementary scales: diurnal/feature correlation and event-window temporal dynamics. This subsection is exploratory governance evidence, not proof of causal physical decomposition. Because routing is regularized by load-balancing objectives, expert activations should be interpreted as correlation-level diagnostics instead of deterministic physical role assignment.

At the diurnal scale, activation shares remain bounded without collapse spikes, which supports auditability of conditional computation. At event windows, routing redistribution is observable and can be logged as a monitoring signal for anomaly triage, but these shifts alone should not be interpreted as direct physical regime identification.

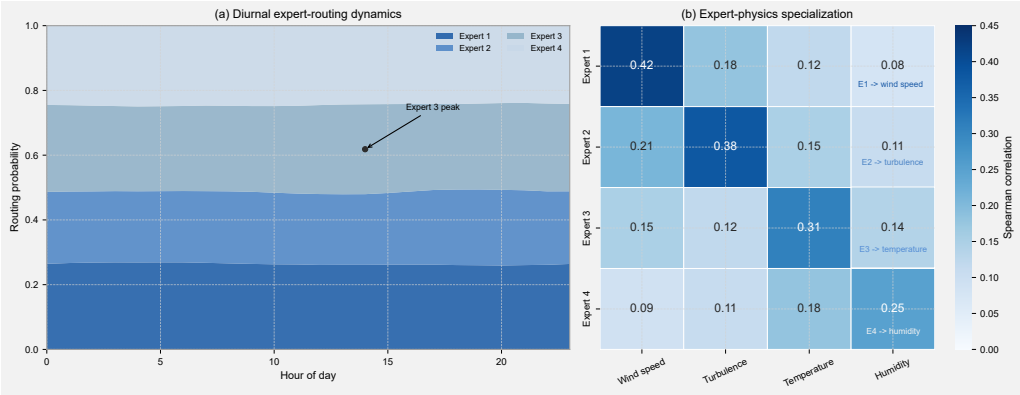

Figure G.12: Interpretability view of sparse routing and expert-feature correlation. The left panel reports diurnal expert-routing composition, and the right panel reports the expert-feature correlation heatmap.

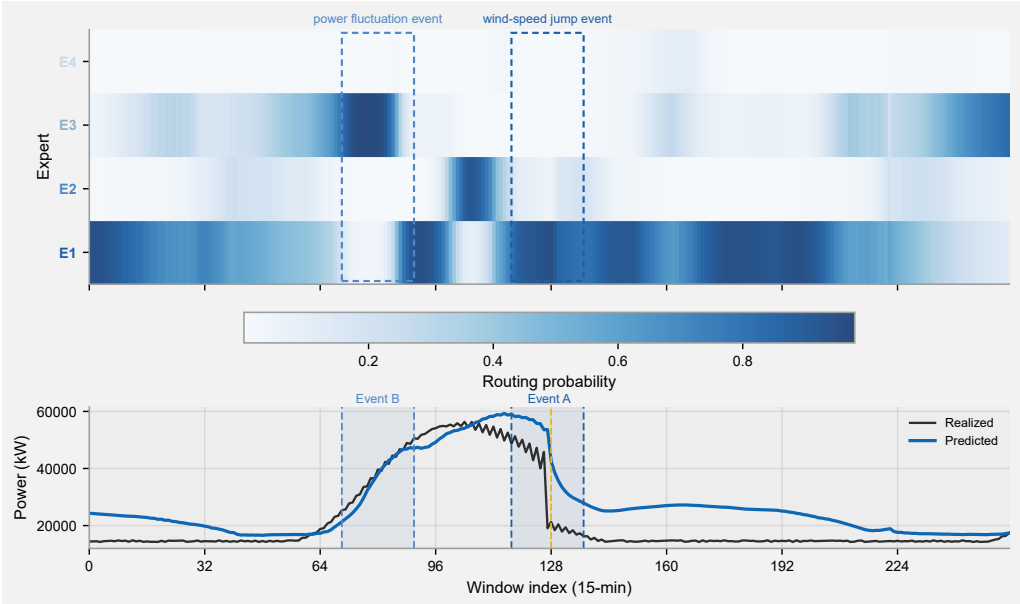

Figure G.13: Time-resolved routing probabilities (4 experts) over a representative 64-hour window with overlaid realized and predicted power. Marked event windows show temporal co-variation between routing redistribution and abrupt trajectory changes; the figure is provided for auditability rather than causal attribution.

Appendix G.7. Statistical significance

We assess whether performance gaps are statistically significant using the Diebold–Mariano (DM) test on squared prediction errors [42]. Let $d_t = \ell_t(\text{Ultra-LSNT}) - \ell_t(\text{Baseline})$ denote the

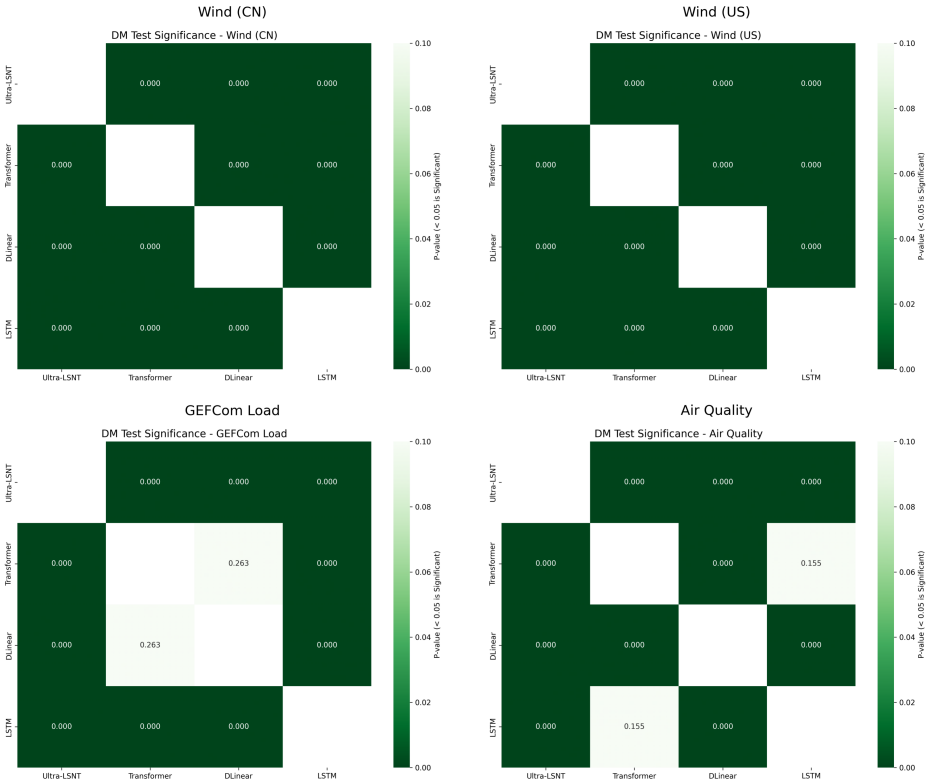

Figure G.14: Direction-aware 2×2 summary of Diebold–Mariano evidence (Ultra-LSNT vs principal baselines) across Wind (CN), Wind (US), GEFCom Load, and Air Quality. Cell entries report direction, p-value, and significance level; complete pairwise DM-statistic magnitude matrices would require dedicated reruns and are not reconstructed here.

per-time step loss differential computed on the same test split; hence DM<0 indicates Ultra-LSNT has smaller average loss, while DM>0 favors the baseline. This significance check is reported to strengthen the auditability of our empirical claims, complementing the descriptive ranking in Table 2. Figure G.14 summarizes the four-domain DM significance matrices used in this revision. Readers should treat the DM view as direction-stability evidence (paired loss differential under fixed splits), not as a replacement for protocol-bound robustness and dispatch screening claims in the main results.

Supplementary DM-test tables indicate statistically significant loss differentials across representative recent backbones. Consistent with Table 2, the differentials favor TimeMixer and iTransformer under clean measurements (DM>0), whereas Ultra-LSNT significantly outper-

forms PatchTST (DM< 0). We also provide scripts and the loss-differential series for the full set of backbones in the supplementary material.

Several recent forecasters can achieve higher peak accuracy under clean measurements (Table 2), but peak clean-data accuracy is not always aligned with stability under realistic sensing imperfections. Accordingly, we report robustness stress tests and downstream dispatch evidence next, where reliability under noise and deployability constraints matter for renewable integration and operational efficiency.

Generative AI and AI-assisted technologies statement

During the preparation of this work, the authors used *ChatGPT* (OpenAI) for language polishing and grammar assistance. The authors performed all scientific interpretation, result reporting, and final manuscript decisions, and they take full responsibility for the content of the publication. No generative AI tools were used to generate or modify experimental data, figures, plots, or graphical abstracts.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] J. Ren, W. Guo, Z. Liu, M. Wu, X. Du, A review of scada systems in power grids: Security, architecture, and perspectives, IEEE Internet of Things Journal 8 (16) (2021) 12741–12759. doi:10.1109/JIOT.2020.3044893.

[2] S. Letzgus, Change-point detection in wind turbine SCADA data for robust condition monitoring with normal behaviour models, Wind Energy Science 5 (2020) 1375–1397. doi:10.5194/wes-5-1375-2020.

[3] I. Vervlimmeren, X. Chesterman, T. Verstraeten, A. Nowé, J. Helsen, Scalable scada-driven failure prediction for offshore wind turbines using autoencoder-based nbm and fleet-median filtering, Wind Energy Science 10 (2025) 2615–2637.

doi:10.5194/wes-10-2615-2025.
URL https://wes.copernicus.org/articles/10/2615/2025/

[4] P. Martín-Calzada, P. Martín Sánchez, F. J. Rodríguez-Sánchez, C. Santos-Pérez, J. Ballesteros, Optimized sensor data preprocessing using parameter-transfer learning for wind turbine power curve modeling, Sensors 25 (17) (2025) 5329. doi:10.3390/s25175329.

[5] H. Khaloie, A. Abdollahi, M. Shafie-khah, A. Anvari-Moghaddam, S. Nojavan, P. Siano, J. P. Catalão, Coordinated wind-thermal-energy storage offering strategy in energy and spinning reserve markets using a multi-stage model, Applied Energy 259 (2020) 114168. doi:10.1016/j.apenergy.2019.114168.

[6] D. Larrahondo, R. Moreno, H. R. Chamorro, F. Gonzalez-Longatt, Comparative performance of multi-period acopf and multi-period dcopf under high integration of wind power, Energies 14 (15) (2021) 4540. doi:10.3390/en14154540.

[7] C. Li, A. Kies, K. Zhou, M. Schlott, O. E. Sayed, M. Bilousova, H. Stöcker, Optimal power flow in a highly renewable power system based on attention neural networks, Applied Energy 359 (2024) 122779. doi:10.1016/j.apenergy.2024.122779.

[8] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, in: Advances in Neural Information Processing Systems, Vol. 34, 2021, pp. 22419–22430.
URL https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html

[9] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting, in: International Conference on Machine Learning, ICML 2022, Vol. 162, 2022, pp. 27268–27286.

[10] Y. Nie, N. H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: Long-term forecasting with transformers, in: The Eleventh International Conference on Learning Representations (ICLR 2023), 2023, pp. 1–15.
URL https://openreview.net/forum?id=Jbdc0vTOcol

[11] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, itransformer: Inverted transformers are effective for time series forecasting, in: The Twelfth International Conference on Learning Representations (ICLR 2024), 2024, pp. 1–16. URL https://openreview.net/forum?id=JePfAI8fah

[12] A. Gu, T. Dao, Mamba: Linear-time sequence modeling with selective state spaces, first Conference on Language Modeling (COLM 2024) (2024). URL https://openreview.net/forum?id=tEYskw1VY2

[13] Y. Zhang, K. Liu, J. Qian, Wind power prediction based on variational mode decomposition and artificial intelligence algorithms: A review, Energy Conversion and Management 227 (2021) 113615. doi:10.1016/j.enconman.2020.113615.

[14] K. Li, Y. Mu, F. Yang, H. Wang, Y. Yan, C. Zhang, A novel short-term multi-energy load forecasting method for integrated energy system based on feature separation-fusion technology and improved cnn, Applied Energy 351 (2023) 121823. doi:10.1016/j.apenergy.2023.121823.

[15] J. Xue, M. Li, Z. Xu, Short-term wind power prediction based on vmd decomposition, kpca dimension reduction and lssvm model with sparrow search algorithm optimization, Energy Reports 6 (2020) 2231–2239. doi:10.1016/j.egyr.2020.08.010.

[16] D. Niu, P. Wang, Z. Zhang, Y. Liu, Wind power forecasting based on ceemdan and lstm optimized by improved sparrow search algorithm, IEEE Access 8 (2020) 177697–177711. doi:10.1109/ACCESS.2020.3027031.

[17] W. Duan, Z. Zhang, A. Zhong, Z. Tang, A robust wind power forecasting framework for non-stationary signals via decomposition and metaheuristic optimization, Energies 18 (24) (2025) 6515. doi:10.3390/en18246515.

[18] A. F. Ansari, L. Stella, C. Türkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, Y. Wang, Chronos: Learning the language of time series, Transactions on Machine Learning Research (2024). URL https://openreview.net/forum?id=gerNCVqqtR

[19] G. Woo, C. Liu, A. Kumar, C. Xiong, S. Savarese, D. Sahoo, Unified training of universal time series forecasting transformers, in: Proceedings of the Forty-first International Conference on Machine Learning (ICML 2024), Vol. 235 of Proceedings of Machine Learning Research, 2024, pp. 53140–53164.
URL https://proceedings.mlr.press/v235/woo24a.html

[20] X. Liu, J. Liu, G. Woo, T. Aksu, Y. Liang, R. Zimmermann, C. Liu, S. Savarese, C. Xiong, D. Sahoo, Moirai-MoE: Empowering time series foundation models with sparse mixture of experts, in: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Vol. 267 of Proceedings of Machine Learning Research, 2025, pp. 38940–38962.
URL https://proceedings.mlr.press/v267/liu25an.html

[21] A. Das, W. Kong, R. Sen, Y. Zhou, A decoder-only foundation model for time-series forecasting, in: Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, PMLR, 2024, pp. 10148–10167.
URL https://proceedings.mlr.press/v235/das24c.html

[22] X. Shi, S. Wang, Y. Nie, D. Li, Z. Ye, Q. Wen, M. Jin, Time-moe: Billion-scale time series foundation models with mixture of experts, international Conference on Learning Representations (ICLR) (2025).
URL https://openreview.net/forum?id=e1wDDFmlVu

[23] T. Burton, N. Jenkins, D. Sharpe, E. Bossanyi, Wind Energy Handbook, 2nd Edition, Wiley, 2011.

[24] International Electrotechnical Commission, IEC 61400-12-1:2017 wind energy generation systems – part 12-1: Power performance measurements of electricity producing wind turbines, International Standard, edition 2.0 (2017).
URL https://webstore.iec.ch/en/publication/26603

[25] B. Stott, J. Jardim, O. Alsac, Dc power flow revisited, IEEE Transactions on Power Systems 24 (3) (2009) 1290–1300. doi:10.1109/tpwrs.2009.2021235.

[26] X. Li, F. Li, Q. Guo, H. Wang, Optimal power flow with high renewable energy

penetration: A review of recent computational methods, Renewable and Sustainable Energy Reviews 189 (2024) 113890. doi:10.1016/j.rser.2023.113890.

[27] H. Khaloie, M. Dolányi, J.-F. Toubeau, F. Vallée, Review of machine learning techniques for optimal power flow, Applied Energy 388 (2025) 125637. doi:10.1016/j.apenergy.2025.125637.

[28] J. Yan, X. Li, H. Wang, F. Si, W. Qiao, Y. Liu, The economic value of wind power forecasting: a data-driven method and its application in various scenarios, Energy 340 (2025) 139151. doi:10.1016/j.energy.2025.139151.

[29] Z. Ma, G. Mei, A hybrid attention-based deep learning approach for wind power prediction, Applied Energy 323 (2022) 119608. doi:10.1016/j.apenergy.2022.119608.

[30] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, G. E. Hinton, Adaptive mixtures of local experts, Neural Computation 3 (1) (1991) 79–87. doi:10.1162/neco.1991.3.1.79.

[31] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean, Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, International Conference on Learning Representations, arXiv:1701.06538. OpenReview ICLR 2017 submission: https://openreview.net/forum?id=B1ckMDqlg (2017). doi:10.48550/arXiv.1701.06538.
URL https://arxiv.org/abs/1701.06538

[32] W. Fedus, B. Zoph, N. Shazeer, Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, Journal of Machine Learning Research 23 (120) (2022) 1–39.
URL https://jmlr.org/papers/v23/21-0998.html

[33] K. Dragomiretskiy, D. Zosso, Variational mode decomposition, IEEE Transactions on Signal Processing 62 (3) (2014) 531–544. doi:10.1109/TSP.2013.2288675.

[34] Z. Wu, N. E. Huang, Ensemble empirical mode decomposition: A noise-assisted data analysis method, Advances in Adaptive Data Analysis 1 (1) (2009) 1–41. doi:10.1142/S1793536909000047.

[35] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, Communications of the ACM 63 (12) (2020) 54–63. doi:10.1145/3381831.
URL https://doi.org/10.1145/3381831

[36] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 3645–3650. doi:10.18653/v1/P19-1355.
URL https://aclanthology.org/P19-1355/

[37] National Energy Administration of China, DL/T 5002-2021 Design code for dispatch automation of regional power grids, DL/T 5002-2021, industry standard issued by the National Energy Administration of China; implemented on 2021-10-26 (in Chinese). (2021).

[38] A. Botterud, Z. Zhou, Use of wind power forecasting in operational decisions, Tech. rep., Argonne National Laboratory, technical report (2011).

[39] C. Draxl, A. Clifton, B.-M. Hodge, J. McCaa, The wind integration national dataset (wind) toolkit, Applied Energy 151 (2015) 355–366.
doi:10.1016/j.apenergy.2015.03.121.

[40] S. De Vito, E. Massera, M. Piga, L. Martinotto, G. Di Francia, On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario, Sensors and Actuators B: Chemical 129 (2) (2008) 750–757.
doi:10.1016/j.snb.2007.09.060.

[41] T. Hong, P. Pinson, S. Fan, H. Zareipour, A. Troccoli, R. J. Hyndman, Global energy forecasting competition 2012, International Journal of Forecasting 30 (2) (2014) 357–363.
doi:10.1016/j.ijforecast.2013.07.001.

[42] F. X. Diebold, R. S. Mariano, Comparing predictive accuracy, Journal of Business & Economic Statistics 13 (3) (1995) 253–263. doi:10.1080/07350015.1995.10524599.